

CASEPATH: AN AGENTIC, PROCESS-FIRST ARCHITECTURE FOR DETERMINING EVIDENCE REQUIREMENTS

Anonymous authors

Paper under double-blind review

ABSTRACT

LLM agents increasingly act in rule-governed workflows, where an action is right only if the process state makes it applicable. In regulated work such as insurance claims, one fact can make a receipt required, irrelevant or premature. Direct generation is then risky: even with the rules in context, one output judges the case and chooses the action, and can mix branches without checkable reasons. CASEPATH, a process-first agentic architecture, splits the work: language models judge conditions, and executable control activates applicable requirements, derives their evidence needs and turns open conditions into questions; each request keeps its reason. When two insurance cases differ in one branch-deciding fact, CASEPATH makes about half as many unjustified checklist changes as direct generation, and about a third as many as a model given CASEPATH’s graph. On synthetic tenancy claims with judgments held fixed, enforcing the enclosing process more than halves the documents requested now and keeps every justified one. Coverage does not rise, as it depends on reading the rules and the case. We release both synthetic datasets, with structured references, for training and evaluating agents. Enforced process state gives agents a control layer between judgment and action.

1 INTRODUCTION

Language-model agents increasingly act inside rule-governed workflows such as customer service (Yao et al., 2025) or claims handling, where an action is right only if the current state of the process makes it applicable. The usual design asks the model for the action directly, sometimes with the rules or a process graph in its prompt. One output must then judge the case and decide what the process requires. It records neither decision, so it can mix requirements from different branches, and nobody can check which actions are justified. Where the process is binding, as in insurance claims, such mistakes are costly: a claimant is asked for documents that do not apply, or a document is dropped without a reason.

We isolate and study one such action: the request for evidence. Suppose a watch is stolen in a household burglary. Should the insurer ask for proof of its value? That depends on a fact the claim category does not reveal: was the watch separately insured above the policy threshold? If it was, its value must be established, and a receipt or an expert’s valuation can show it. If it was not, neither document is needed, and if nobody knows yet, the insurer should first ask. Each answer puts the case on a different branch, so the same receipt can be required, irrelevant or premature. A document on file can also miss the point: a photograph shows that the watch existed, not what it was worth.

A document matters only because an active requirement needs a fact that it can establish. Judging the facts is a question about the case, and deciding what the case requires is a question about the process, so CASEPATH gives the two to different components (Figure 1). Language models read the rules and judge the case. Executable process control decides which requirements are active and what evidence each needs, turns open conditions into questions and keeps each request’s reason.

We contribute in four ways:

1. **Architecture.** We introduce CASEPATH, a process-first control layer between language-model judgment and agent action that derives every request or question, with its reason, from the process state (Section 3, Algorithm 1).

2. **Evaluation.** We report two tests that compare process control with common designs, such as direct generation and a process graph in the prompt (Section 4). In Study A (insurance), two cases differ in one fact that decides a branch, and only the documents that branch governs should change. In Study B (tenancy), with the model’s judgments held fixed, enforcing the enclosing process should remove requests that do not apply and keep the justified ones.
3. **Datasets.** We release the synthetic cases of both studies for training and evaluating agents: Study A’s 36 pairs of theft cases, each with the documents its one changed fact requires, and Study B’s 150 tenancy claims, each pairing a customer message and any attachments with a reference of process, obligations, document states, next action and sources (Figures 3 and 4).
4. **Empirical insight.** Process control makes evidence requests more selective. In Study A, CASEPATH cuts unjustified checklist changes by almost half against the Direct baseline and by almost two thirds against Graph context, which has CASEPATH’s graph in its prompt (Table 2). In Study B, inherited conditions more than halve the requests made now and keep every justified one (Figures 7 and 8). Coverage does not rise, as it rests on reading the rules and the case (Tables 2 and 5). Every request carries its chain, so each Study A error traces to its source, most often a rule that can be corrected once (Appendix B.3).

The benchmarks, saved outputs and evaluators behind every number are released.¹

2 RELATED WORK

Rule and case-management languages (OASIS LegalRuleML Technical Committee, 2021; Merigoux et al., 2021; Object Management Group, 2024; 2016; Vaculín et al., 2011; López et al., 2020) and proactive enforcement (Basin et al., 2024) encode or apply rules, but none reads a free-text case. PAGED and WorFBench test extracting or writing procedures (Du et al., 2024; Qiao et al., 2025), and neuro-symbolic systems have a model formalise a problem for an exact solver (Pan et al., 2023; Guan et al., 2023). In CASEPATH a model judges each case, and code applies the process.

Process-aware agents follow a workflow or policy in the prompt (Xiao et al., 2024; Yao et al., 2025), a hand-built state machine (Wu et al., 2024), guidance from a procedural graph (Lu et al., 2026) or a compiled procedure paged frame by frame (Yu et al., 2026), and guardrails check another agent’s actions (Chen et al., 2025; Xiang et al., 2025). In all of them a model picks the actions, though StateFlow fixes a few in advance; in CASEPATH code decides every request from the model’s judgments. The closest work, an evidence ledger (Kim, 2026), also holds a branch’s obligations until its condition resolves, but a model plans them per query, and it names no inherited conditions.

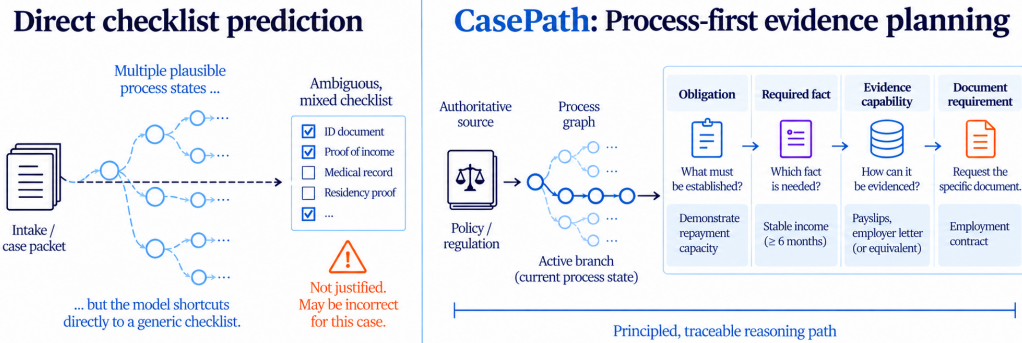


Figure 1: **CASEPATH derives each document from the active process.** On a loan application: direct prediction can mix branches; CASEPATH follows the active one from obligation to fact, evidence and document.

¹Code and data: <https://anonymous.4open.science/r/casepath-9673/>.

Algorithm 1 CASEPATH: the model judges the case; code derives every question and request.
 $\wedge_3$ is three-valued: false if any input is false, else unresolved if any is unresolved, else true.

Input: process built once from the rules: branches b and obligations o with conditions c_b and c_o ; each o needs facts f , and each f has routes ρ (sets of documents); a case file

Output: questions; document requests, each with its chain $o \rightarrow f \rightarrow \rho$

- 1: $v(c) \leftarrow$ true, false or unresolved for every condition c , quoting the case ▷ model
- 2: judge each document on file: adequate or not ▷ model
- 3: **for** each obligation o **do**
- 4: $a_o \leftarrow v(c_o) \wedge_3 v(c_b)$ for every branch b enclosing o ▷ full process scope
- 5: **if** a_o is false **then** skip o ▷ irrelevant
- 6: **else if** a_o is unresolved **then** ask what would settle its unresolved conditions ▷ premature
- 7: **else for** each fact f of o **do** ▷ required
- 8: $\rho^* \leftarrow$ the route of f with the fewest missing or inadequate documents
- 9: request those documents of ρ^* , each with chain $o \rightarrow f \rightarrow \rho^*$

Why these agents are not our baselines. None takes a process and a case and returns document requests: most return actions, some need inputs our cases lack, such as a hand-built state machine or another agent’s actions, and some have no public code (Appendix E). Our baselines test their shared ideas: Direct reads the sources as a τ -bench agent reads its policy, Graph context adds CASEPATH’s graph as FlowBench adds a workflow, every Study B baseline reads the compiled process, Rule-first has the model write and run its own rules, and Evidence-first and Document-first list evidence needs first, like the ledger’s planner. We did not rebuild their runtimes or claim to beat them.

Information-seeking agents retrieve (Asai et al., 2024) or ask (Li et al., 2024; Zhang et al., 2024; Taranukhin et al., 2026) to complete an answer or a plan, and legal work benchmarks evidence reasoning in litigation (Bi et al., 2026) or assesses compliance (Chung et al., 2025). CASEPATH ties each request to an active obligation and each question to an open condition.

3 ARCHITECTURE

The rules become a process of nested branches; each has a condition and holds requirements, or *obligations*. An obligation needs a *required fact*, such as the watch’s value. An *evidence capability*, here proof of value, can establish that fact. Each allowed set of documents that supplies it, such as the receipt or an expert’s valuation, is a *route*, and one route is enough. Where the rules are not yet formal, agents read the source texts into this structure. For each case they mark every condition true, false or unresolved, quoting the case, and judge the documents on file.

A *planner* in code does the rest (Algorithm 1; formal version in Appendix A.1). An obligation applies only if its own condition holds and so does that of every branch enclosing it, which we call *full process scope*. Proof of the watch’s value is thus required, irrelevant or premature when its condition is true, false or unresolved. Checking the enclosing branches can hold or drop a request that the obligation’s own condition allows, and can never add one. A timing condition can also hold a request back. For each fact, the planner picks the route with the fewest missing or inadequate documents and requests only those, so each request keeps its chain from source to document.

In Figure 2, a tenant reports mould in the bedroom and attaches the email that notified the landlord. Suppose a landlord notified of a defect must assess and address it. The email proves the notice but not how far the mould has spread, the one fact still open. Photographs or an inspection report can show it, so the tenant is asked for one.

Software. We release CASEPATH with this paper as agentic claims-handling software.² It is a local workbench for handling Study B’s 150 claims, and it also hosts Study A’s method, which reproduces all 72 recorded cases from their saved judgments. The results below measure the method, not the software (Appendix Table 16).

²Software: <https://anonymous.4open.science/r/casepath-9673/casepath/>.

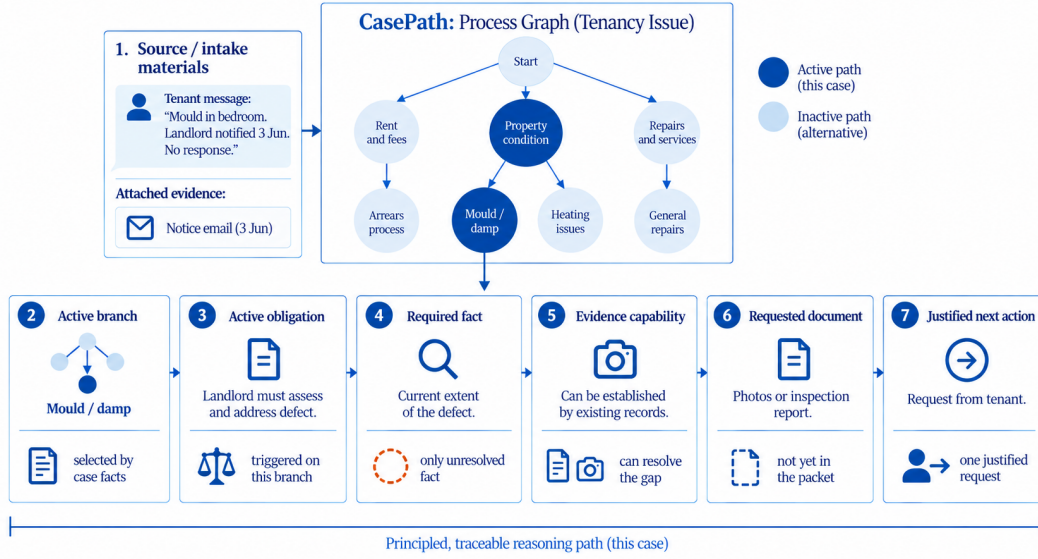


Figure 2: **The one open fact decides what to request.** An illustrative process: the case facts select the mould branch (dark nodes), and the rule is hypothetical.

4 METHODOLOGY

Process control makes two predictions, and each study tests one (Table 1). **RQ1:** when a branch-deciding fact changes, do the requests change where that branch requires and nowhere else? **RQ2:** with the model’s judgments held fixed, do conditions inherited from enclosing branches remove requests that do not apply and keep justified ones? The studies share no data or code (Appendix A). We release both datasets under CC BY 4.0, with datasheets and verification scripts (Figures 3 and 4; Appendix D). Their cases are synthetic; the law, policy terms and forms they use are real.

4.1 STUDY A: ONE CHANGED FACT

Agents built the rules from federal law (Swiss Confederation, 2024), four insurers’ policy terms (CSS Versicherung AG, 2024; AXA Versicherungen AG, 2021; Generali Allgemeine Versicherungen AG, 2026; Simpego Versicherungen AG, 2024), a claim form (CSS Versicherung AG, 2026) and industry model conditions (Schweizerischer Versicherungsverband SVV, 2022), turning 308 statements quoted from 197 passages into conditional rules over a fixed catalogue of documents. Study A runs a variant of Algorithm 1: each rule checks only its own condition, and every route of every active rule is requested (Appendix A.3). We took 9 branch conditions from the sources, such as whether a stolen item was separately insured, and set each in four neutral household-theft stories. Each pair’s reference lists the justified document changes. Code derived it from a contract, an agent-built record of what the sources support; no legal expert reviewed the contract (Appendix D.1). Figure 3 shows the 36

Table 1: **Each study tests one prediction of process control.**

	Study A	Study B
Question	RQ1: does one changed fact cause the right checklist change?	RQ2: does the enclosing process remove requests that do not apply?
Cases	Household-theft insurance claims	Tenancy disputes
Rules come from	Agents reading law and policy texts	Code compiling synthetic templates
What is scored	Changes between the checklists of two cases	Complete lists of requests
Form of Algorithm 1	Own conditions only, every route	Inherited conditions, one route

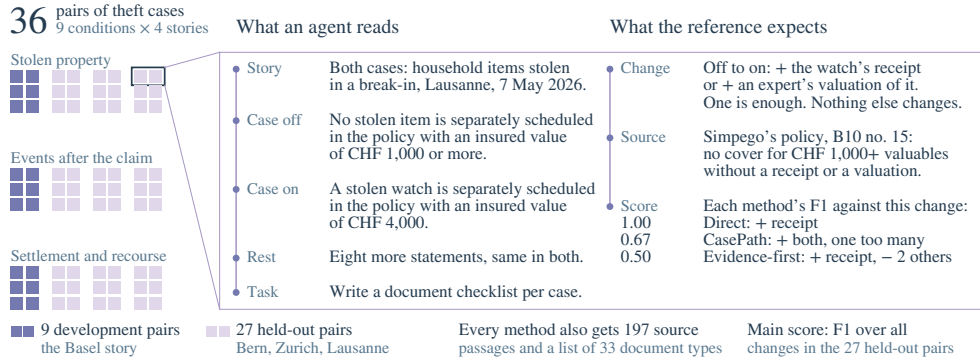


Figure 3: A Study A pair tells one theft story twice with one condition changed, and its reference lists only the documents that this change justifies. Rows are the nine conditions and columns the four stories, with two squares per pair, one per case (dark: development); the outlined pair (scheduled valuable, Lausanne story) is opened at right.

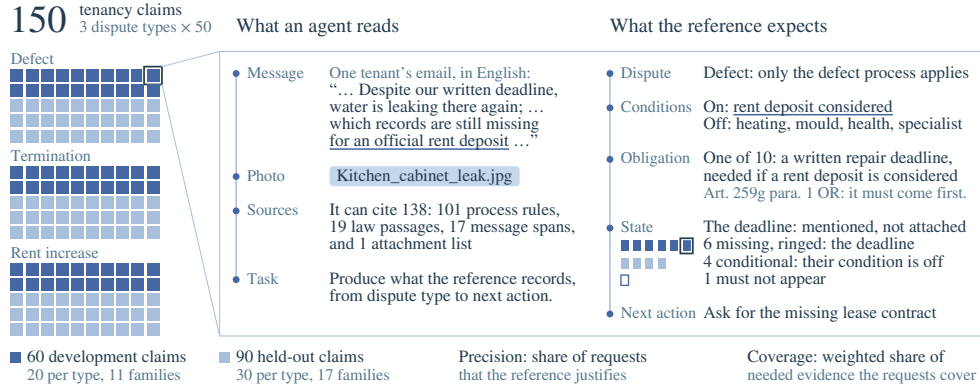


Figure 4: A Study B claim is one tenant's message, and its reference runs from the dispute type to the next action. Each square is one claim (dark: development). A family groups near-duplicate claims, and no family crosses the split. Claims arrive by email, portal, chat or phone note; 75 are in English and 75 in Swiss Standard German. The outlined defect claim is opened at right and followed through one of its ten obligations, whose document is outlined in the state row; the next action applies to the whole claim. The underlined words in the message set the underlined condition.

released pairs³ and opens one: a short theft story followed by nine condition statements, told twice with one statement changed. When a stolen watch is separately insured for CHF 4,000, the reference adds a receipt or an expert's valuation (either is enough) and changes nothing else. Development used one story's 9 pairs; the other 27 were read once, after the method, data, analysis and criteria were frozen.

A *checklist change* is a document added or withdrawn between the two checklists of a pair. It is *correct* if the sources justify it and *wrong* if not, and a justified change not made is *missed*; *change F1* pools these counts over the held-out pairs. Three baselines share the model (openai/gpt-5.6-terra, temperature 0), sources, catalogue and cases but write the checklists themselves: *Direct* from the case and the sources, *Graph context* with CASEPATH's graph and rules in its prompt, and *Evidence-first* after listing the evidence it needs and retrieving passages. Appendix B gives the statistics, the eight predeclared criteria and the costs. A wrong document on both checklists never shows up as a change, so Study B scores complete lists.

³Datasets: <https://anonymous.4open.science/r/casepath-9673/data/>.

4.2 STUDY B: SAME JUDGMENTS, WITH AND WITHOUT THE ENCLOSING PROCESS

In Study B, code compiles the process from three synthetic tenancy templates, agents only judge the case, and the planner applies full process scope (Algorithm 1). The one condition an obligation inherits is the claim’s dispute type (a defect, a termination or a rent increase), so a document that only a defect dispute needs is requested only in a claim judged a defect. With every timing condition true and one route per capability, Study B does not test timing or route choice (Appendix A.2).

The benchmark has 150 synthetic claims from 28 families of near-duplicate claims; 60 served for development and 90 were held out. Figure 4 opens one. An agent reads the tenant’s message and any attachments; the reference records the dispute type, the branches that apply, the chain from each obligation to the document that meets it, the state of each document and the next action. In the opened claim the tenant mentions a written repair deadline without attaching it, and the reference marks that document missing. Five pipelines share the process, case file, model (`openai/gpt-5.6-terra`) and 2 calls per claim. CASEPATH assembles its output in code from the model’s judgments; four baselines have the model write it, directly (Direct) or after first drafting the documents (Document-first), the process branches (Process-context) or condition-and-action rules (Rule-first). Two controls reuse CASEPATH’s saved judgments: *Local scope* drops the inherited conditions, and *Compiled control* recomputes the same decisions in another form and matches exactly.

The benchmark’s own evaluator accepted no output, as its reference scenarios leave the pipelines’ condition symbols unassigned, so none of the 12 predeclared targets was met (Appendix C). Without new model calls, we therefore compare CASEPATH with Local scope at the same saved judgments and score every pipeline’s requested documents against the reference (*document-request scoring*). A request is *valid* if its chain from obligation to fact to evidence matches the reference. *Justified-request precision* is the share of requests that are valid, and *critical-evidence recall*, or *coverage*, is the weighted share of required evidence that a complete requested route covers. The held-out claims were seen during product work, so they are not a fresh test (Appendix C.5).

5 RESULTS

5.1 STUDY A: CASEPATH CHANGES FEWER DOCUMENTS WITHOUT A REASON

Against Direct, CASEPATH makes 12 fewer wrong changes (15 against 27) at the cost of one correct change (24 against 25), and it gets the most pairs exactly right (Table 2). Eleven of the 12 errors it avoids are withdrawals. No pair requires one, so the benchmark cannot credit a correct withdrawal, and CASEPATH makes none (Figure 5).

All 120 wrong changes of the baselines are on documents no branch governs, against 9 of CASEPATH’s 15 (Figure 6A). The baselines write each checklist afresh, so such documents drift between the two cases. CASEPATH’s changes follow the condition: reassigned to other conditions, their change F_1 falls from 0.667 to at most 0.083 (Figure 6B), and each of its 39 changes links to its condition.

CASEPATH is exact on every event after the claim; averaged over the insurer’s deadline, its declination and the recovery of stolen items, no baseline exceeds 0.40 (Table 3), and Direct never asks for the declination letter. CASEPATH’s chains explain its low scores on settlement and recourse: the model misread the expert-procedure request, the planner requested every route, and broad rules added two

Table 2: **CASEPATH makes the fewest wrong checklist changes and gets the most pairs exactly right.** The 27 held-out pairs require 33 changes, all additions; brackets: 95% intervals; bold: best.

Method	Checklist changes			Exact pairs ↑	Change F_1 [95% interval] ↑
	Correct ↑	Wrong ↓	Missed ↓		
CASEPATH	24	15	9	12/27	0.667 [0.435, 0.880]
Direct	25	27	8	9/27	0.588 [0.404, 0.759]
Graph context	31	41	2	4/27	0.590 [0.471, 0.716]
Evidence-first	16	52	17	2/27	0.317 [0.239, 0.400]



Figure 5: **The baselines’ extra errors are mostly withdrawals, which no pair requires.** (A) F_1 over all changes, 95% intervals. (B) Correct additions, wrong additions and wrong withdrawals.

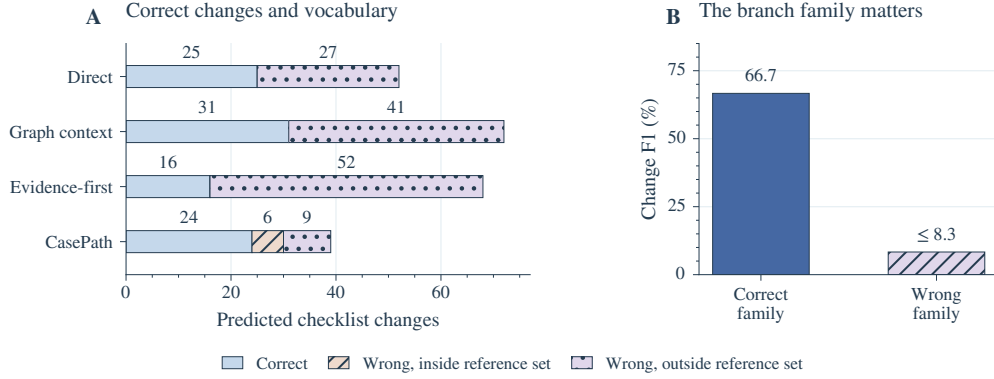


Figure 6: **The baselines change documents that the changed condition cannot justify.** (A) Correct and wrong changes; the reference set holds documents some branch governs. (B) CASEPATH’s change F_1 with the correct condition (labelled family) and its best over 5,008 reassignments.

Table 3: **CASEPATH is exact on every event after the claim and weakest on settlement and recourse.** Mean pair F_1 over the three held-out stories of each condition; bold: best method.

	Branch condition	CASEPATH	Direct	Graph context	Evidence-first
Stolen property	Scheduled valuable	0.67	1.00	0.69	0.50
	Bicycle or e-bike	1.00	0.61	0.82	0.21
	Misused SIM card	0.67	0.78	0.61	0.39
Events after the claim	Insurer’s deadline	1.00	0.17	0.39	0.23
	Insurer’s declination	1.00	0.00	0.28	0.22
	Items recovered	1.00	0.80	0.52	0.39
Settlement and recourse	Formal expert procedure	0.00	0.49	0.44	0.56
	Repair needing consent	0.50	0.78	0.89	0.00
	Third party at fault	0.00	0.89	0.80	0.33
Mean over the nine conditions		0.65	0.61	0.60	0.31

consents for a repair and kept the third party’s documents on both checklists. Three quarters of its errors lie in the rules the agents wrote (Appendix B.3).

On total error, CASEPATH is cheapest unless a miss costs more than $26/7 \approx 3.7$ wrong changes; above that, Graph context, which misses 7 fewer changes but makes 26 more wrong ones, is cheapest (Appendix Figure 10).

Table 4: CASEPATH meets five of eight predeclared criteria, none of them for accuracy.

	Criterion	Target	CASEPATH	Met
Accuracy	Smallest F_1 margin over a baseline	≥ 0.10	0.076	no
	Largest Holm-corrected paired p	≤ 0.05	0.797	no
	Precision of the changes	≥ 0.70	0.62	no
Traceability	Changes traced to a changed condition	≥ 0.90	1.00	yes
	Required changes reachable by a source chain	≥ 0.80	0.82	yes
	Requested documents with a source chain	all	all	yes
Specificity	Changes per required change	0.75–1.50	1.18	yes
	Correct- vs. wrong-condition F_1	>97.5 th pct.	0.667 vs. ≤ 0.083	yes

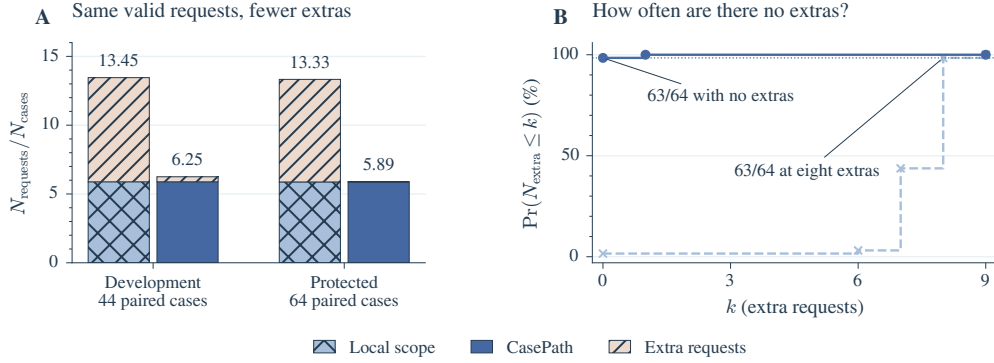


Figure 7: **Full process scope removes unnecessary requests and keeps the valid ones.** (A) Requests per claim on the 108 claims planned both ways (64 held out, labelled protected); blue: valid; peach: unnecessary (labelled extra). (B) Held-out claims with at most k unnecessary requests.

Table 5: **Counting every claim, full process scope more than doubles justified-request precision and keeps coverage.** All 150 claims at the saved judgments; failed outputs score zero.

Method	Claims with output	Justified-request precision $\uparrow$	Critical-evidence recall $\uparrow$
CASEPATH	110 of 150	0.721	0.641
Local scope	108 of 150	0.323	0.630

All five traceability and specificity criteria are met, and the three accuracy criteria are not (Table 4). The margin over the closest baseline (0.076) and the precision of the changes (0.62) fall short of their targets, and the paired test over nine conditions finds no significant gain; against Direct, the losses on settlement and recourse offset most of the gains on the events after the claim.

5.2 STUDY B: THE ENCLOSING PROCESS REMOVES REQUESTS THAT DO NOT APPLY

On the 108 claims planned both ways, the inherited conditions cut requests from 1,445 to 652, and all 635 valid ones remained (Figure 7). Every removed request belonged to another dispute type: 710 were held because the model left that type open, and 83 were dropped because it judged the type false. On 63 of the 64 held-out claims, CASEPATH made no unnecessary request. Over all claims, justified-request precision rises from 0.323 under Local scope to 0.721, and coverage holds (Table 5).

Deleting as many requests at random would keep under half of the valid ones (Figure 8B). In the output graphs, full process scope also removes over half of the nodes and relations without losing a reference match (Figure 8A).

CASEPATH also produces a usable list far more often: its output, assembled by code, is scoreable for a weighted 73.6% of claims against at most 46.7% for the model-written baselines, and the lists that exist score similarly across pipelines (Appendix Figure 13). Conventional paired contrasts favour

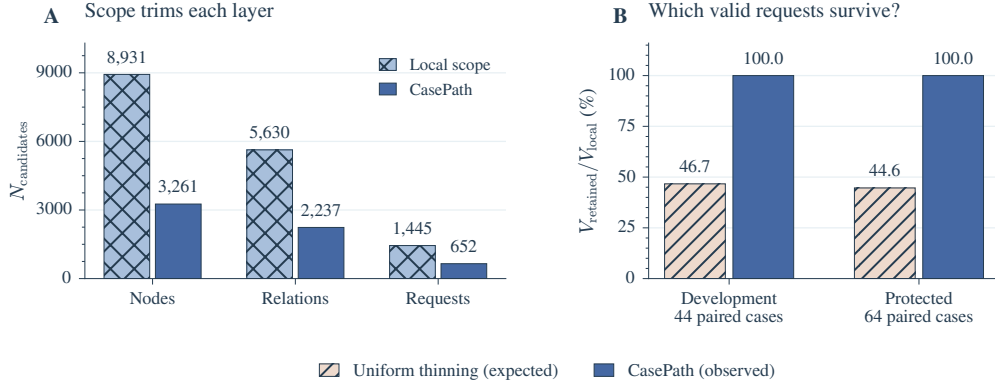


Figure 8: **Full process scope prunes every layer without losing a reference match.** (A) Nodes, relations and requests per scope. (B) Valid requests kept versus random deletion; protected: held out.

CASEPATH; the conservative ones, which count even a baseline’s failure against it, are all negative, so under the predeclared rule it beats no baseline (Appendix Table 12).

6 DISCUSSION

Graph context had CASEPATH’s graph and rules in its prompt and still made 41 wrong changes, more than Direct, all on documents no branch governs. Our results favour letting the process decide which actions are admissible and asking the model only for the judgments that code cannot make.

Information-seeking agents decide when to retrieve or what to ask; rule-governed work asks instead whether an active obligation needs the evidence now. When process state decides, each request has a reason, and an open fact, such as whether the watch was separately insured, becomes a question first. In Study B, CASEPATH held back 2,006 documents until a condition was resolved and the baselines rarely did (Appendix Table 11); no study scores whether a hold is right.

Limitations. Study A meets none of its three accuracy criteria, and under Study B’s predeclared rule CASEPATH beats no baseline. Study B’s evidence comes from rescoring saved runs, since the benchmark’s evaluator accepted none; its held-out claims were seen during product work, and its process has one inherited condition. Of Study A’s nine misses, six sit in rules as the agents wrote them, and in three, requesting every route keeps the document on both checklists. CASEPATH separates reading from control, so the rules and the planner can each be improved on their own. We built both datasets, and neither study shows legal correctness in real cases; expert review, deeper processes, real claims and other jurisdictions come next.

7 CONCLUSION

CASEPATH enforces in code the process state that makes an action applicable; in both studies its requests were more selective, though not more complete, and each kept its reason. The released datasets let others train and test agents that ask what a case needs before requesting a document.

REPRODUCIBILITY STATEMENT

The anonymous supplement holds everything needed to check the results offline: for Study A, the frozen sources, rules, planner, benchmark, predictions, provider receipts and analysis code; for Study B, the 150 claims with their reference answers, the compiled process, the prediction code, the original evaluator, the document-request scorer and every recorded run, failures included. Each reported result is linked to its source file by hash, and the reproduction commands recompute every analysis without network or model access (Appendices B and C.5). Both datasets are also released

on their own in the repository’s data/, each with its licence, datasheet and verification script (Appendix D). New model outputs would need provider access and are not deterministic.

ETHICS STATEMENT

The studies use public sources and synthetic cases that contain no personal data. The released datasets are synthetic apart from quoted law, policy and claim-form passages and official cantonal form templates; third-party text stays with its issuers and is excluded from the data licence. The evaluated Study B messages named a real insurer’s intake mailbox as their recipient; the release replaces it with a fictional address of the same byte length (Appendix D). CASEPATH is a research method for planning evidence requests; it gives no legal advice and makes no automated coverage decisions. Real use would require the correct policy and jurisdiction, professional review, privacy controls and ways to correct or appeal a decision.

AI USE STATEMENT

The authors conceived the research questions, designed the CASEPATH architecture and both studies, and directed the work throughout. In this work, we used generative AI tools as assistants for the following tasks with required disclosure: implementing, testing and running code; generating the synthetic cases of both datasets and Study A’s reference contract, including translating German source passages, to our specifications; reformatting and pseudonymising the datasets for release; categorising Study A’s errors by their origin; and giving feedback on our framework, hypotheses, methodology and interpretation of results. We have not used them to formulate mathematical claims or to write proofs or their ingredients; no other task with required disclosure applies to this work. We also used generative AI tools for literature search and figures and for drafting and editing text. We have reviewed all AI-assisted work: code was tested, every reported number was recomputed from the released files, and every reference was checked against its source. We take responsibility for the final content of this work, including text, claims and artifacts produced with the aid of generative AI.

REFERENCES

- Akari Asai, Zeqiu Wu, Yizhong Wang, Avi Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun (eds.), *International Conference on Learning Representations*, volume 2024, pp. 9112–9141, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/file/25f7be9694d7b32d5cc670927b8091e1-Paper-Conference.pdf.
- AXA Versicherungen AG. Allgemeine Vertragsbedingungen (AVB) Haushaltversicherung: Privathaftpflicht, Hausrat, Cyber Plus, Zusatzversicherungen und Services. Policy terms, 16162DE, 2021. URL <https://www.axa.ch/servlets/external/docstoredocument?accesscode=ag58y>. Ausgabe 10.2021. Accessed 24 September 2026.
- David Basin, Søren Debois, and Thomas Hildebrandt. Proactive enforcement of provisions and obligations. *Journal of Computer Security*, 32(3):247–289, 2024. doi: 10.3233/JCS-210078.
- Sheng Bi, Kai Yu, Jiaqi Li, Xiaoyan Wang, Fuhui Sun, Zeyi Miao, Jing Wang, Xin Geng, and Lusheng Wang. CLER: A benchmark for Chinese litigation evidence reasoning. *Information Processing & Management*, 63(5):104667, 2026. doi: 10.1016/j.ipm.2026.104667. URL <https://www.sciencedirect.com/science/article/pii/S0306457326000592>.
- Samuel R. Bowman and George Dahl. What will it take to fix benchmarking in natural language understanding? In *Proceedings of NAACL-HLT*, pp. 4843–4855, 2021. doi: 10.18653/v1/2021.naacl-main.385.
- Zhaorun Chen, Mintong Kang, and Bo Li. ShieldAgent: Shielding agents via verifiable safety policy reasoning. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu (eds.), *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning*

- Research*, pp. 8313–8344. PMLR, 2025. URL <https://proceedings.mlr.press/v267/chen25ae.html>.
- Jiseong Chung, Ronny Ko, Wonchul Yoo, Makoto Onizuka, Sungmok Kim, Tae-Wan Kim, and Won-Yong Shin. GraphCompliance: Aligning policy and context graphs for LLM-based regulatory compliance. *arXiv preprint arXiv:2510.26309*, 2025. URL <https://arxiv.org/abs/2510.26309>.
- CSS Versicherung AG. Hausratversicherung: Allgemeine Versicherungsbedingungen (AVB). Policy terms, Form. 519d-01.24-eff, 2024. URL https://www.css.ch/content/dam/css/de/documents/privatkunden/richtig-versichert/avb/519_d_avb_hausratversicherung.pdf. Ausgabe 02.2024. Accessed 24 September 2026.
- CSS Versicherung AG. Schadenanzeige Hausrat / Gebäude. Claim notification form, Form. 15d-08.26-pdf, 2026. URL https://www.css.ch/content/dam/css/de/documents/privatkunden/richtig-versichert/formulare/15_d_formular_schadenanzeige_hausrat.pdf. Accessed 24 September 2026.
- Shehzaad Dhuliawala, Mojtaba Komeili, Jing Xu, Roberta Raileanu, Xian Li, Asli Celikyilmaz, and Jason Weston. Chain-of-verification reduces hallucination in large language models. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 3563–3578, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.212. URL <https://aclanthology.org/2024.findings-acl.212/>.
- Weihong Du, Wenrui Liao, Hongru Liang, and Wenqiang Lei. PAGED: A benchmark for procedural graphs extraction from documents. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 10829–10846, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.583. URL <https://aclanthology.org/2024.acl-long.583/>.
- Robert Geirhos, Jørn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2:665–673, 2020. doi: 10.1038/s42256-020-00257-z.
- Generali Allgemeine Versicherungen AG. Allgemeine Versicherungsbedingungen (AVB) Hausratversicherung. Policy terms, 2026. URL <https://www.generali.ch/content/dam/generali/pdf/produkt dokumente/avb-ki/hausrat-haftpflicht/avb-haushaltversicherung-de.pdf>. Ausgabe 2026. Accessed 24 September 2026.
- David Graus. From legal text to executable decision models: Evaluating structured representations for legal decision model generation. *arXiv preprint arXiv:2604.17153*, 2026.
- Lin Guan, Karthik Valmeekam, Sarath Sreedharan, and Subbarao Kambhampati. Leveraging pre-trained large language models to construct and utilize world models for model-based task planning. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (eds.), *Advances in Neural Information Processing Systems*, volume 36, pp. 79081–79094. Curran Associates, Inc., 2023. doi: 10.52202/075280-3459. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/f9f54762cbb4fe4dbffdd4f792c31221-Paper-Conference.pdf.
- Suchin Gururangan, Swabha Swayamdipta, Omer Levy, Roy Schwartz, Samuel R. Bowman, and Noah A. Smith. Annotation artifacts in natural language inference data. In *Proceedings of NAACL-HLT*, pp. 107–112, 2018. doi: 10.18653/v1/N18-2017.
- Dexter Hadley. CANONIC: Governance is compilation. *arXiv preprint arXiv:2607.05410*, 2026.
- Divyansh Kaushik and Zachary C. Lipton. How much reading does reading comprehension require? A critical investigation of popular benchmarks. In *Proceedings of EMNLP*, pp. 5010–5015, 2018. doi: 10.18653/v1/D18-1546.

- Onur Selim Kilic, Yeti Z. Gurbuz, Cem O. Yaldiz, Afra Nawar, Etrit Haxholli, Oğul Can, and Eli Waxman. Guideline2Graph: Profile-aware multimodal parsing for executable clinical decision graphs. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, pp. 6942–6951, June 2026. URL https://openaccess.thecvf.com/content/CVPR2026W/Med-Reasoner/html/Kilic_Guideline2Graph_Profile-Aware_Multimodal_Parsing_for_Executable_Clinical_Decision_Graphs_CVPRW_2026_paper.html.
- Myung Ho Kim. Evidence-obligation pool-gated retrieval: Stable multi-cycle retrieval via evidence ledger and warrant judge. Preprints.org, 2026. URL <https://www.preprints.org/manuscript/202607.1060>. Version 1, posted 15 July 2026; not peer reviewed.
- Dawei Li, Renliang Sun, Yue Huang, Ming Zhong, Bohan Jiang, Jiawei Han, Xiangliang Zhang, Wei Wang, and Huan Liu. Preference leakage: A contamination problem in LLM-as-a-judge. In *International Conference on Learning Representations*, 2026. arXiv:2502.01534.
- Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-Bank: A comprehensive benchmark for tool-augmented LLMs. In *Proceedings of EMNLP*, pp. 3102–3116, 2023. doi: 10.18653/v1/2023.emnlp-main.187.
- Shuyue Stella Li, Vidhisha Balachandran, Shangbin Feng, Jonathan S. Ilgen, Emma Pierson, Pang Wei Koh, and Yulia Tsvetkov. MediQ: Question-asking LLMs and a benchmark for reliable interactive clinical reasoning. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 28858–28888. Curran Associates, Inc., 2024. doi: 10.52202/079017-0908. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/32b80425554e081204e5988ab1c97e9a-Paper-Conference.pdf.
- Hugo A. López, Søren Debois, Tijs Slaats, and Thomas T. Hildebrandt. Business process compliance using reference models of law. In *Fundamental Approaches to Software Engineering*, pp. 378–399, 2020. doi: 10.1007/978-3-030-45234-6_19.
- Andrew Lou and David Shin. Legal reasoning is not lawyering: Rethinking legal benchmarks for pro se access to justice. *arXiv preprint arXiv:2606.23716*, 2026.
- Yuxing Lu, Yicheng Chen, Shanchan Wu, and Sercan Ö. Arık. Procedural graphs: Self-evolving execution structures for LLM agents. *arXiv preprint arXiv:2609.09153*, 2026.
- R. Thomas McCoy, Ellie Pavlick, and Tal Linzen. Right for the wrong reasons: Diagnosing syntactic heuristics in natural language inference. In *Proceedings of ACL*, pp. 3428–3448, 2019. doi: 10.18653/v1/P19-1334.
- Masha Medvedeva and Pauline McBride. Legal judgment prediction: If you are going to do it, do it right. In *Proceedings of the Natural Legal Language Processing Workshop 2023*, pp. 73–84, 2023. doi: 10.18653/v1/2023.nllp-1.9.
- Jacob Menick, Maja Trebacz, Vladimir Mikulik, John Aslanides, Francis Song, Martin Chadwick, Mia Glaese, Susannah Young, Lucy Campbell-Gillingham, Geoffrey Irving, and Nat McAleese. Teaching language models to support answers with verified quotes. *arXiv preprint arXiv:2203.11147*, 2022. URL <https://arxiv.org/abs/2203.11147>.
- Denis Merigoux, Nicolas Chataing, and Jonathan Protzenko. Catala: A programming language for the law. *Proceedings of the ACM on Programming Languages*, 5(ICFP):1–29, 2021. doi: 10.1145/3473582. URL <https://arxiv.org/abs/2103.03198>.
- OASIS LegalRuleML Technical Committee. LegalRuleML core specification version 1.0. OASIS Standard, 2021. URL <https://docs.oasis-open.org/legalruleml/legalruleml-core-spec/v1.0/os/legalruleml-core-spec-v1.0-os.html>.
- Object Management Group. Case management model and notation (CMMN), version 1.1. OMG Specification, 2016. URL <https://www.omg.org/spec/CMMN/1.1>.
- Object Management Group. Decision model and notation (DMN), version 1.5. OMG Specification, 2024. URL <https://www.omg.org/spec/DMN/1.5>.

- Liangming Pan, Alon Albalak, Xinyi Wang, and William Wang. Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning. In Houda Bouamor, Juan Pino, and Kalika Bali (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 3806–3824, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.248. URL <https://aclanthology.org/2023.findings-emnlp.248/>.
- Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The Berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu (eds.), *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 48371–48392. PMLR, 13–19 Jul 2025. URL <https://proceedings.mlr.press/v267/patil25a.html>.
- Adam Poliak, Jason Naradowsky, Aparajita Haldar, Rachel Rudinger, and Benjamin Van Durme. Hypothesis only baselines in natural language inference. In *Proceedings of *SEM*, pp. 180–191, 2018. doi: 10.18653/v1/S18-2023.
- Shuofei Qiao, Runnan Fang, Zhisong Qiu, Xiaobin Wang, Ningyu Zhang, Yong Jiang, Pengjun Xie, Fei Huang, and Huajun Chen. Benchmarking agentic workflow generation. In *International Conference on Learning Representations*, volume 2025, pp. 69679–69703, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/adbe936993aa7cf41e45054d8b72f183-Abstract-Conference.html.
- Schweizerischer Versicherungsverband SVV. Allgemeine Bedingungen (AVB) für die Hausratversicherung. Non-binding model policy conditions (unverbindliche Musterbedingungen), Ausgabe 2022, 2022. URL <https://svv.ch/sites/default/files/media/documents/2024-02/AVB%20Hausratversicherung%202022.pdf>. Version 01.04.2022. Accessed 24 September 2026.
- ServiceNow. Add a document list definition to a service definition. Financial Services Operations documentation, 2026. URL <https://www.servicenow.com/docs/r/financial-services-operations/insurance-claims/add-document-list-definition-to-service-definition.html>. Brazil release; updated September 10, 2026. Accessed 24 September 2026.
- Simpego Versicherungen AG. Simpego Haushaltversicherung Home: Allgemeine Versicherungsbedingungen (AVB). Policy terms, 2024. URL https://simpego.ch/dam/AVB_Simpego_Home_de.pdf. Ausgabe Dezember 2024. Accessed 24 September 2026.
- Saku Sugawara, Kentaro Inui, Satoshi Sekine, and Akiko Aizawa. What makes reading comprehension questions easier? In *Proceedings of EMNLP*, pp. 4208–4219, 2018. doi: 10.18653/v1/D18-1453.
- Swiss Confederation. Bundesgesetz vom 2. April 1908 über den Versicherungsvertrag (Versicherungsvertragsgesetz, VVG), SR 221.229.1. Fedlex, Swiss Federal Chancellery, 2024. URL https://www.fedlex.admin.ch/eli/cc/24/719_735_717/20240101/de. Consolidated version of 1 January 2024 (Stand am 1. Januar 2024), German text; Arts. 38–41a, 45 and 46 used. Accessed 24 September 2026.
- Swiss Confederation. Bundesgesetz vom 30. März 1911 betreffend die Ergänzung des Schweizerischen Zivilgesetzbuches (Fünfter Teil: Obligationenrecht), SR 220. Fedlex, Swiss Federal Chancellery, 2026. URL https://www.fedlex.admin.ch/eli/cc/27/317_321_377/20260101/de. Consolidated version of 1 January 2026 (Stand am 1. Januar 2026), German text; tenancy provisions Arts. 257d–273 used. Accessed 24 September 2026.
- Maksym Taranukhin, Shuyue Stella Li, Evangelos Milios, Geoff Pleiss, Yulia Tsvetkov, and Vered Schwartz. InfoGatherer: Principled information seeking via evidence retrieval and strategic questioning. *arXiv preprint arXiv:2603.05909*, 2026. URL <https://arxiv.org/abs/2603.05909>.

- Paul Thomas, Nick Craswell, Mark Sanderson, Seth Spielman, Robert Sim, and Ryen W. White. Evaluation validity in information retrieval. In *Proceedings of SIGIR*, pp. 2734–2746, 2026. doi: 10.1145/3805712.3808538.
- Roman Vaculín, Richard Hull, Terry Heath, Craig Cochran, Anil Nigam, and Piyawadee Sukaviriya. Declarative business artifact centric modeling of decision and knowledge intensive business processes. In *2011 IEEE 15th International Enterprise Distributed Object Computing Conference*, pp. 151–160, 2011. doi: 10.1109/EDOC.2011.36.
- Yiran Wu, Tianwei Yue, Shaokun Zhang, Chi Wang, and Qingyun Wu. StateFlow: Enhancing LLM task-solving through state-driven workflows. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=3nTbuygoop>.
- Zhen Xiang, Linzhi Zheng, Yanjie Li, Junyuan Hong, Qinbin Li, Han Xie, Jiawei Zhang, Zidi Xiong, Chulin Xie, Carl Yang, Dawn Song, and Bo Li. GuardAgent: Safeguard LLM agents via knowledge-enabled reasoning. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu (eds.), *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 68316–68342. PMLR, 2025. URL <https://proceedings.mlr.press/v267/xiang25a.html>.
- Ruixuan Xiao, Wentao Ma, Ke Wang, Yuchuan Wu, Junbo Zhao, Haobo Wang, Fei Huang, and Yongbin Li. FlowBench: Revisiting and benchmarking workflow-guided planning for LLM-based agents. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 10883–10900, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-emnlp.638. URL <https://aclanthology.org/2024.findings-emnlp.638/>.
- Chen Yang, Ruping Xu, Ruizhe Li, Bin Cao, and Jing Fan. Business as rulesual: A benchmark and framework for business rule flow modeling with LLMs. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 35198–35220. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-long.1625.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. In Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu (eds.), *International Conference on Learning Representations*, volume 2025, pp. 9965–10017, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/file/1b126cc38b8638e07bef37e7b2bb72bf-Paper-Conference.pdf.
- Chenglin Yu, Li Yin, Qingxin Fan, Ying Yu, Runyang Ray Zhong, and Ming Li. Compile, then page: Executable SOP programs and a capability-gated runtime for procedural LLM agents, 2026. URL <https://arxiv.org/abs/2607.11346v3>. Version 3, 23 July 2026.
- Yu Yuan, Lili Zhao, Kai Zhang, Guangting Zheng, and Qi Liu. Do LLMs overcome shortcut learning? An evaluation of shortcut challenges in large language models. In *Proceedings of EMNLP*, pp. 12188–12200, 2024. doi: 10.18653/v1/2024.emnlp-main.679.
- Xuan Zhang, Yang Deng, Zifeng Ren, See-Kiong Ng, and Tat-Seng Chua. Ask-before-Plan: Proactive language agents for real-world planning. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 10836–10863, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-emnlp.636. URL <https://aclanthology.org/2024.findings-emnlp.636/>.

A THE PLANNER IN DETAIL

Algorithm 1 sketches the planner. This appendix states it formally in the same notation and then gives the form each study runs: Study B runs the full planner on a process in which every obligation inherits one condition, and Study A runs a reduced form in which each rule checks only its own condition and every route is requested. Table 6 sets the two studies side by side; they share no sources, cases or code, and we do not compare their planners.

A.1 THE PLANNER

For each obligation, the planner decides whether it applies, whether its evidence may be requested now and which route should supply that evidence. In Figure 2, the obligation to assess and address the mould applies only if its own condition holds (the landlord has been told) and so does the condition of every branch above it. The model judges each condition c for the case at hand, and its judgment $v(c) \in \{T, F, U\}$ is true, false or unresolved. A branch b has a condition c_b and parent branches $\text{pa}(b)$. An obligation o sits in branch $b(o)$ and has its own condition c_o and a timing condition t_o . The planner computes

$$\begin{aligned} q_b &= v(c_b) \wedge_3 J_b(\{q_p\}_{p \in \text{pa}(b)}), \\ a_o &= q_{b(o)} \wedge_3 v(c_o), \quad r_o = a_o \wedge_3 v(t_o). \end{aligned} \quad (1)$$

The first line decides whether branch b applies. It checks the branch’s own condition and, through its parents, every branch above it; J_b says whether all or any of the parents must apply, and a root has no parent requirement. The second line decides whether the obligation applies (a_o) and whether its evidence may be requested now (r_o). Here $\wedge_3$ is the three-valued conjunction defined in Algorithm 1. Conditions may also use negation $\neg$, which swaps true and false, and disjunction $\vee$, the dual of $\wedge_3$. The planner requests an obligation’s documents now when $r_o = T$ and drops them when $a_o = F$. Otherwise it holds them and asks about any condition that is unresolved (Appendix C.3). Line 4 of Algorithm 1 computes a_o when every branch requires all of its parents and leaves out the timing condition.

The Local scope control of Section 4.2 cuts every link to the branches above and keeps everything else. It sets $q_b = v(c_b)$ for every branch, so each obligation checks only its own condition and that of the branch that holds it. The conditions of the branches above that branch, which we call inherited, drop out. With the values ordered $F < U < T$, $\wedge_3$ returns the lowest of its inputs, so the inherited conditions can only lower a_o and r_o . Full scope can therefore hold or drop a request that Local scope makes now, and it can never add one.

Each obligation o needs a set of evidence capabilities $\mathcal{E}_o$. Capability e establishes the required fact f_e , and $\mathcal{R}_e$ is its set of routes, any one of which is enough. For the documents on file H , the model judges whether each is present and adequate. A route ρ is satisfied, written $\rho(H) = T$, only if its documents are present, adequate and sufficient together. Let $M_{e\rho}$ be the documents of ρ that are missing or inadequate, and $V_{e\rho}$ the items whose presence or adequacy needs review. For each capability of an obligation with $r_o = T$, the planner chooses

$$\rho_e^* = \arg \min_{\rho \in \mathcal{R}_e}^{\text{lex}} (\mathbf{1}\{\rho(H) \neq T\}, |M_{e\rho}|, |V_{e\rho}|, \text{sort}(M_{e\rho}), \text{id}(\rho)) \quad (2)$$

and requests $\bigcup_{o: r_o=T} \bigcup_{e \in \mathcal{E}_o} M_{e\rho_e^*}$. It prefers a route the file already satisfies, then the route with fewer missing or inadequate documents, then the one with fewer items to review; sorted document identifiers and the route identifier break ties. Algorithm 1 indexes routes by the fact f_e that a capability establishes, so its ρ^* is ρ_e^* . Routes are chosen one capability at a time, and a capability without any route is recorded as an evidence gap. The next action follows a fixed order: ask a question, review evidence, request documents, review an open gap, and only then take a permitted case action.

A.2 STUDY B’S PROCESS

Code compiles Study B’s process deterministically from the three tenancy templates (Section 4.2). The only condition an obligation inherits is the claim’s dispute type, which the model judges from the case. Every timing condition t_o is true, so $r_o = a_o$, and each capability has one route with one

document, so ρ_e^* is that route. Study B therefore tests inherited applicability, the holding of documents for unresolved branches and the state of documents on file, while the timing and route rules run with nothing to decide.

Table 6: **The two studies share the planner’s design but no sources, cases or code.** Study A runs the reduced planner of Appendix A.3, and Study B the full planner of Appendix A.1.

	Study A (insurance)	Study B (tenancy)
<i>Planner</i>		
Rules come from	Law and policy texts, which agents turn into evidence rules mapped to document routes	Three synthetic tenancy templates, which code compiles; every method uses the result
Request activated by	The rule’s own condition; an unresolved one becomes a question	The obligation’s own condition and the inherited dispute type; an unresolved one becomes a question
Documents on file	Left out without an adequacy check (no case has any)	Each judged for presence and adequacy
Routes requested	Every route of every capability	The chosen route; each capability has one route with one document
When the case changes	The plan is recomputed from the edited case	The plan is recomputed from new judgments
<i>Evaluation</i>		
Cases	9 development and 27 held-out pairs over nine branch conditions; 72 cases	60 development and 90 held-out claims from 11 and 17 families covering defects, terminations and rent increases
Model calls	One call per condition over all cases of a split for CASEPATH; one call per case for Direct and Graph context, two for Evidence-first	Two per claim for each of the five pipelines; the two controls reuse CASEPATH’s judgments
What is scored	Checklist changes between the two cases of a pair	Complete outputs (the benchmark’s evaluator) and requested documents (document-request scoring)
Statistics	Bootstrap over conditions, exact paired tests and a wrong-condition control; the three accuracy criteria fail	Weighted descriptive contrasts; no significance claim

A.3 STUDY A’S PLANNER

Study A runs the smallest planner that can respond to a changed branch condition: it checks only each rule’s own condition. A Study A rule plays the role of an obligation, and we write it o . In the notation of Appendix A.1, $a_o = r_o = v(c_o)$, and $a_o = r_o = \top$ for a rule without a condition. A rule is active when $a_o = \top$ and yields a question when $a_o = \perp$. For each capability of an active rule, the planner requests every route, together with the routes that would confirm each condition judged true. It leaves out documents on file without checking their adequacy, and the larger process graph plays no part. With $O(x)$ the rules active in case x and $\mathcal{R}_c^{\text{conf}}$ the routes that confirm condition c , the planner requests

$$D(x, H) = \left[\bigcup_{o \in O(x)} \bigcup_{e \in \mathcal{E}_o} \bigcup_{\rho \in \mathcal{R}_e} \rho \cup \bigcup_{c: v(c) = \top} \bigcup_{\rho \in \mathcal{R}_c^{\text{conf}}} \rho \right] \setminus H. \quad (3)$$

No Study A case has documents on file, so we write $D(x) = D(x, \emptyset)$. Each request keeps its rule, required fact, evidence capability and source quotation. Every correction to the rules was made from the sources alone and stays in the record.

Because every route contributes, the planner never chooses among alternatives. A rule without a condition puts all of its routes on every checklist. The experts’ appointment, one route of the rule on determining the loss, is on both checklists of every pair, so turning on the formal expert procedure cannot add it (three misses in Table 7). Even if the rule for a scheduled valuable had listed the receipt and the expert valuation as two routes instead of one, the planner would have requested both.

B STUDY A IN DEPTH

Section 5.1 reports Study A as counts of checklist changes. This appendix shows how a pair is scored, why the baselines change more documents and where each of CASEPATH’s 24 errors arose; it ends with the statistics and the cost of each method.

B.1 RULES, FREEZE AND SCORING

Each of the 308 statements that the agents extracted from the 197 passages of Section 4.1 (Table 14) is covered by a rule or marked as requiring no evidence. The frozen representation that the agents built from the sources holds 98 evidence rules, 13 statements that require no evidence, 87 case conditions and 33 document types. Completeness means covering these statements; other law that could apply is out of scope.

Table 3 abbreviates the nine branch conditions. In the *stolen property* group they are a scheduled valuable at or above the threshold defined in the sources, a stolen bicycle or e-bike and theft of a SIM card with a claim of misuse. The *events after the claim* are a written demand with a warning of forfeiture and a deadline, the insurer’s declination and the recovery of stolen items or news of them. *Settlement and recourse* covers the choice of a formal expert procedure, a planned repair above the consent threshold and an identifiable third party who caused the loss.

Before the held-out pairs were read, a hash-bound freeze fixed the sources, the benchmark with its two amendments made before evaluation, the extracted rules, the map from evidence to catalogue documents, the condition interpreter (the model step that judges conditions) and the planner. The same freeze fixed the model, provider and temperature settings, the analysis code, the development split, the baselines’ development predictions and the method’s tests. We implemented the baselines ourselves; none reproduces a published system (Appendix E). Development used 9 pairs (18 cases), one story per condition. Once predeclared checks on these pairs passed, we read the other 27 pairs (54 cases) once, under a separate freeze. The eight criteria of Table 4 were fixed before that read, and no file changed after it. They were not filed with an external registry; the hash-bound freeze in the supplement records them.

A pair is scored by the documents a method adds and withdraws between its two cases. Take pair i , whose branch condition is off in case x^- and on in case x^+ , and let $D(x)$ be the checklist a method returns for case x . The method’s signed change for the pair is

$$\Delta(D) = \{+d : d \in D(x^+) \setminus D(x^-)\} \cup \{-d : d \in D(x^-) \setminus D(x^+)\}. \quad (4)$$

When the sources allow alternatives, the pair has several acceptable references $G \in \mathcal{G}_i$, one for each combination of alternatives. The scorer compares the prediction $P_i = \Delta(D)$ with each of them and keeps the one that maximises, in lexicographic order,

$$(F_1(P_i, G), |P_i \cap G|, \text{Prec}(P_i, G), \text{Rec}(P_i, G), -|G|, \text{sort}(G));$$

the last entry breaks any remaining tie. Each reference holds one alternative, so asking for both when one suffices counts as a wrong change. Precision and recall are zero when their denominators are empty, and F_1 is one when both the prediction and the reference are empty. Change F_1 pools the counts over the 27 held-out pairs (Table 2), and pair F_1 is averaged within each condition (Table 3). Every reference change is an addition, so the benchmark rewards a method that withdraws nothing and cannot test whether a withdrawal is correct.

B.2 WHY THE BASELINES CHANGE MORE DOCUMENTS

The baselines change more documents because nothing ties the two checklists of a pair together. Each baseline writes the checklist of each case afresh, whereas CASEPATH derives both from the judged conditions through the same rules. Figure 9 shows the effect for every condition. It splits each method’s changes by the *reference vocabulary*, labelled reference set in Figure 6: every document that some branch of the benchmark governs, that is, every document in a reference annotation or in an acceptable signed change. On almost every condition, the baselines add or drop general claim documents outside the vocabulary, such as the claim form or the payment details. CASEPATH’s nine changes outside the vocabulary are the repair consents and the SIM bank statement of Table 7, and its other six wrong changes use documents that the branches govern. The split reuses the reference the scorer chose for each pair and changes no score.

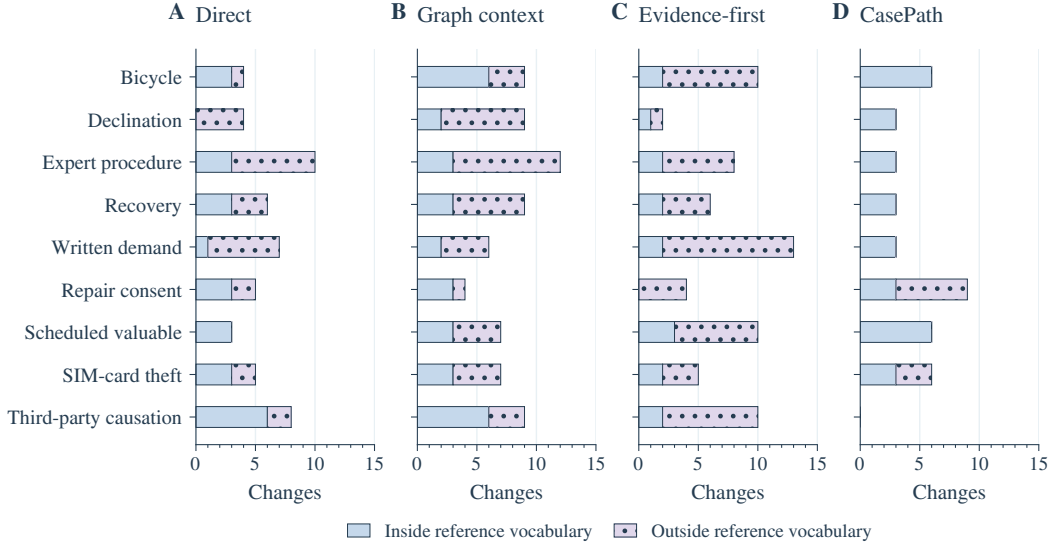


Figure 9: **The baselines leave the reference vocabulary on almost every condition; CASEPATH does so on two.** Changes by method and branch condition over the three held-out stories of each. Row labels abbreviate Table 3; written demand is the insurer’s deadline. A change inside the vocabulary can still be wrong, in its sign or its branch.

B.3 WHERE CASEPATH’S ERRORS COME FROM

Eighteen of CASEPATH’s 24 errors lie in the rules as the agents wrote them, where each can be found through its chain and corrected once. The 24 errors are its 15 wrong changes and 9 misses, all on five of the nine conditions; the bicycle and the three events after the claim have none. We located each error by following its request to the rule behind it and to the quotation behind the model’s judgment, and Table 7 groups the errors by where they arose. In nine of the eighteen rule errors, a condition is broader than the reference and the model’s judgment follows its wording; in the other nine, a route or rule asks for too much. The remaining six come from a misread case and from the planner’s policy of requesting every route (Appendix A.3). The nine misses share one mechanism: each missed document was already on the checklist before the condition changed, so the change had nothing to add.

The break-even cost of a miss (Section 5.1) depends mostly on one condition, the third party at fault. With a wrong change costing 1 and a missed change costing $\lambda \geq 0$, the counts of Table 2 give the total errors

$$\begin{aligned} L_{\text{CasePath}} &= 15 + 9\lambda, & L_{\text{Direct}} &= 27 + 8\lambda, \\ L_{\text{Graph}} &= 41 + 2\lambda, & L_{\text{Evidence}} &= 52 + 17\lambda. \end{aligned}$$

CASEPATH is cheapest for $\lambda < (41 - 15)/(9 - 2) = 26/7$ and Graph context above that point; Direct and Evidence-first are never cheapest (Figure 10). Six of CASEPATH’s nine misses come from the third party at fault, whose two required documents were already on both checklists. Without those six, and with the other counts unchanged, the crossing point would move from 3.7 to 26. The calculation prices all documents alike and uses observed counts without an uncertainty estimate.

B.4 STATISTICS AND CONTROLS

Both the intervals and the paired test treat each of the nine conditions as one unit. The 95% intervals come from 10,000 bootstrap draws that resample the conditions with replacement (seed 20260917), each keeping its three held-out stories. The exact one-sided paired test enumerates all 512 sign assignments of the nine per-condition differences, so it can reach significance only when the gains are consistent across conditions. Its p -values against Direct, Graph context and Evidence-first are 0.436, 0.398 and 0.043, and 0.797, 0.797 and 0.129 after Holm correction.

Table 7: **Three quarters of CASEPATH’s errors in Study A trace to rules as the agents wrote them.** Wrong and missed changes over the three held-out stories of each condition, grouped by where the error arose; the number after each group is its count of errors.

Condition	Changes		What happened
	Wrong	Missed	
<i>In the rules: a condition broader than the reference (9)</i>			
Repair needing consent	6	0	The condition on preserving evidence covers any planned alteration of damaged property, so a repair adds the insurer's and the police's consent
Third party at fault	0	3	The condition on recourse covers any third party, so the unidentified thief keeps the assignment of claims on both checklists
<i>In the rules: a route or rule that asks for too much (9)</i>			
Scheduled valuable	3	0	One route asks for the receipt and the expert valuation together, where the sources accept either
Misused SIM card	3	0	The rule's only route adds a bank statement to the theft report
Third party at fault	0	3	The claim-form section on the liable person comes from a rule without a condition, so it is on both checklists
<i>In reading the case (3)</i>			
Formal expert procedure	3	0	A written request for the procedure is read as one to appoint an expert, which adds the insurer's written demand
<i>In the planner: every route requested (3)</i>			
Formal expert procedure	0	3	Requesting every route keeps the experts' appointment on both checklists
Total	15	9	

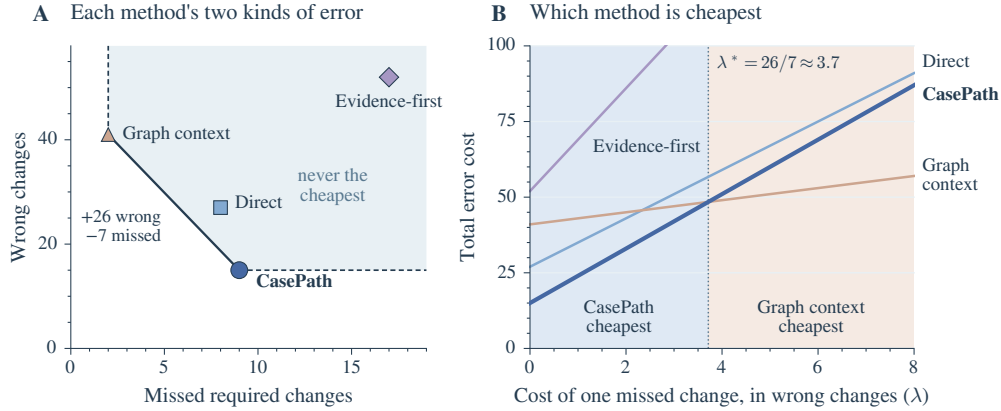


Figure 10: **CASEPATH has the lowest total error unless a missed change costs more than 3.7 wrong ones.** (A) Wrong against missed changes; methods in the shaded region are never the cheapest. (B) Total error when a wrong change costs 1 and a missed change λ .

Two controls test whether the changes depend on the case. An input-free control fits the best constant change on eight conditions and scores it on the ninth; its highest F_1 is 0.0, so a fixed answer learned from other conditions scores nothing. The wrong-condition control of Figure 6B reassigns the saved changes to other conditions by 8 cyclic shifts and 5,000 random derangements, with the add-one p -value $(1 + N_{\geq}) / (1 + 5,008)$, where $N_{\geq}$ counts reassignments scoring at least the observed F_1 . No reassignment reaches the observed F_1 of 0.667, so $N_{\geq} = 0$; the maximum of 0.083 is taken over the reassignments alone. A lookup from conditions to documents would pass both controls, which is why Tables 3 and 7 examine the changes themselves.

B.5 COST

CASEPATH spends every call on judging conditions and leaves the checklist to code. Each of its calls judges one condition for all 54 held-out cases at once, while a baseline call reads one case. Table 8 lists the provider receipts of the held-out run, and Figure 11 sets them against quality. CASEPATH reads fewer tokens than Direct and writes about five times as many. Among the baselines, prompt size explains most of the differences: Evidence-first reads only twelve retrieved passages, while Graph context reads the process graph and the rules on top of the sources.

Table 8: **CASEPATH costs more than Direct and less than Graph context, with more calls than either.** Held-out run; every method uses `openai/gpt-5.6-terra` at temperature 0 with one logged attempt per call. Evidence-first allows 18,000 completion tokens for extraction and 14,000 for planning. Charges exclude source preparation.

Method	Calls	Tokens			Cost (USD)
		Prompt	Completion	Completion limit per call	
CASEPATH	87	1,005,211	243,891	10,000	5.44
Direct	54	2,900,625	45,678	14,000	4.83
Graph context	54	5,345,421	42,663	14,000	8.41
Evidence-first	108	259,890	66,450	18,000/14,000	1.06

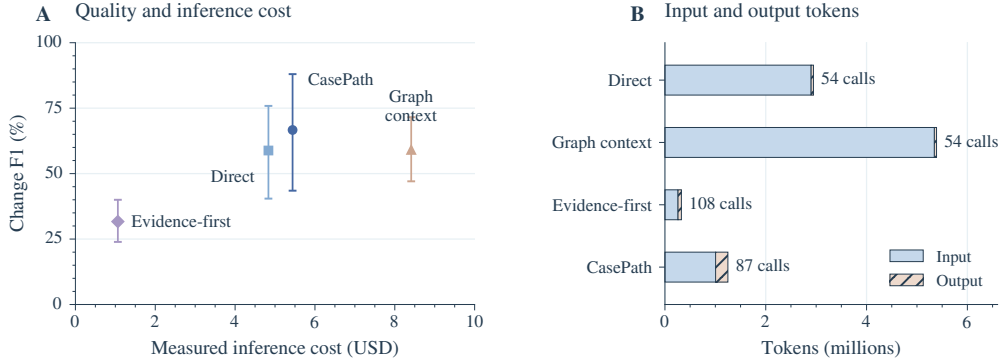


Figure 11: **CASEPATH has the highest change F_1 at a cost between Direct and Graph context.** (A) Provider charge and change F_1 with 95% intervals. (B) Input (prompt) and output (completion) tokens, with the number of calls.

C STUDY B IN DEPTH

The benchmark’s own evaluator accepted no Study B output, so every Study B result in Section 5.2 comes from analysing the saved runs without new model calls. Here we explain that failure and the rescoring, then break down what full process scope holds, drops and keeps when the model’s judgments are fixed.

C.1 RUNS AND THE ORIGINAL EVALUATOR

A run is one method applied to one claim; the five pipelines and two controls on 150 claims make 1,050 runs, the cells of Figure 12. Each pipeline makes 2 model calls per claim, each with medium reasoning effort and at most 4,096 completion tokens, and uses no tools and no resampling. A run records either a validated output or the provider’s output with the reason it failed validation. Most failures are of five kinds: a relation points to a node the output does not contain; a completion reaches the token limit or ends without a final answer; an index does not resolve in the shared public codebook; a document’s presence contradicts its judged state; or a control is blocked because the CASEPATH run it depends on failed. The rest fail other checks, such as incomplete judgments or a

document outside the judged route. The four baselines must write the whole output in two calls and fail on more than half of their claims; CASEPATH, whose output is assembled by code, fails on 40 (Figure 12A). Every failure keeps its provider charges, and no repaired output replaces a failed run.

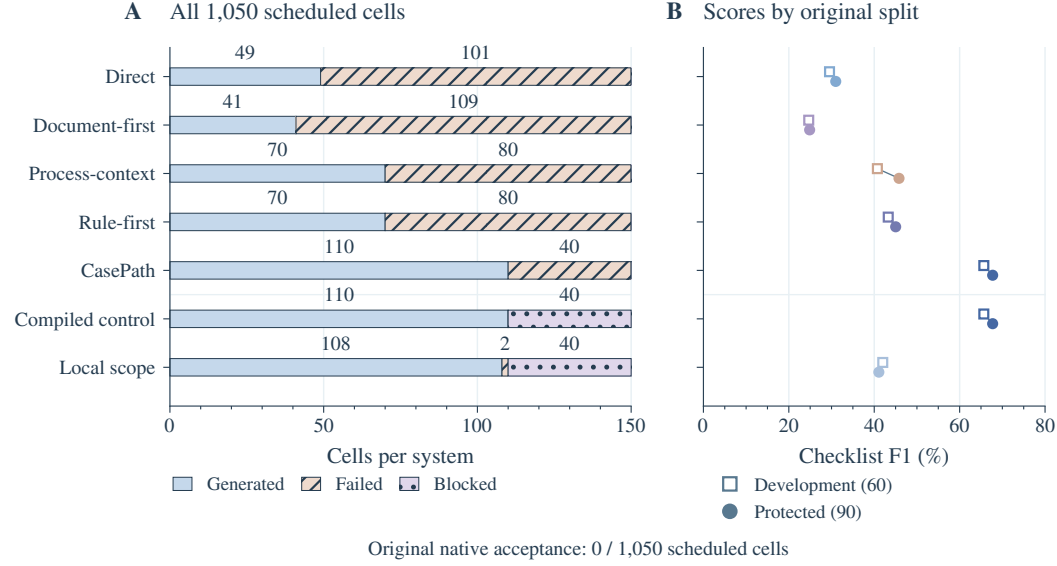


Figure 12: **Every run stays in the record, and scores are similar on both splits.** (A) Generated, failed and blocked runs per method, with the controls below the line; the benchmark’s evaluator accepted none. (B) Checklist F_1 from document-request scoring by split, with failed runs scored as zero; *protected* marks the held-out claims.

The benchmark’s evaluator accepts an output only after evaluating the output’s conditions against the claim’s reference scenario, but every output uses the 50 condition symbols that the public compiler introduces for all methods, and the reference scenarios assign none of them. All 558 generated outputs therefore fail the check, all 1,050 runs receive the failure penalty, and none of the 12 predeclared targets for the held-out claims is met. The symbols cannot simply be renamed, because the two sides read an absent condition differently. In a counterexample written without looking at any target, CASEPATH’s planner returns unresolved for an absent condition and the evaluator’s parser raises a missing-variable error; with null substituted, the evaluator returns false while the planner still returns unresolved. A renamed symbol, a default value or an injected predicted state would each silently turn “not yet known” into “false”, so neither analysis below uses any of them.

C.2 RESCORING THE REQUESTS

Metrics. A request is valid when its whole chain, from obligation to required fact, evidence capability and document, matches the reference. An empty list has justified-request precision one, which is why the rate is read together with coverage. Coverage weights each piece of required evidence by its criticality and counts it only when a complete route for it is present and its obligation is active. Document-request scoring compares the distinct documents P that a method requests now with the claim’s reference documents G . It reports precision $|P \cap G|/|P|$, recall $|P \cap G|/|G|$, the checklist $F_1 = 2|P \cap G|/(|P| + |G|)$ and the unnecessary fraction $|P \setminus G|/|P|$. When both sets are empty, F_1 is one; an empty prediction has precision one and an unnecessary fraction of zero, and an empty reference has recall one.

Eligibility and failed outputs. Under the predeclared procedure a request list is scored only once the evaluator accepts the whole output, and no list qualified. Document-request scoring changes only this eligibility rule. It takes each output’s unchanged list of documents requested now, maps their labels through the original public mapping and applies the original scorer, without evaluating any condition against the reference. Each of the 150 references gives one eligible set of documents. The 558 generated lists receive observed scores, and the 492 failed or blocked outputs keep the frozen

worst case: zero for a metric where higher is better and one where lower is better. No request is invented for a missing output, and a missing reference label never shrinks a denominator.

Weights and contrasts. Claim i has weight $\omega_i = 1/(3 m_i n_i)$, where m_i is the number of families of its dispute type and n_i the number of claims in its family; the three dispute types thus count equally, and so do the families within a type. Availability A is the weighted share of claims with a scoreable list and quality Q the weighted mean F_1 over those claims, so overall $F_1 = AQ$ (Figure 13). Let s_{ik}^{CP} and s_{ik}^j be the scores of CASEPATH and baseline j on claim i for metric k , and $\sigma_k \in \{-1, 1\}$ the metric’s benefit direction. The conventional paired contribution of claim i against baseline j is $\sigma_k(s_{ik}^{\text{CP}} - s_{ik}^j)$; the conservative one equals it when both scores exist and is -1 otherwise. Under the predeclared rule, a baseline counts as beaten only if the conservative contrasts in checklist F_1 , critical-evidence recall and the unnecessary fraction all exceed their thresholds over all held-out claims.

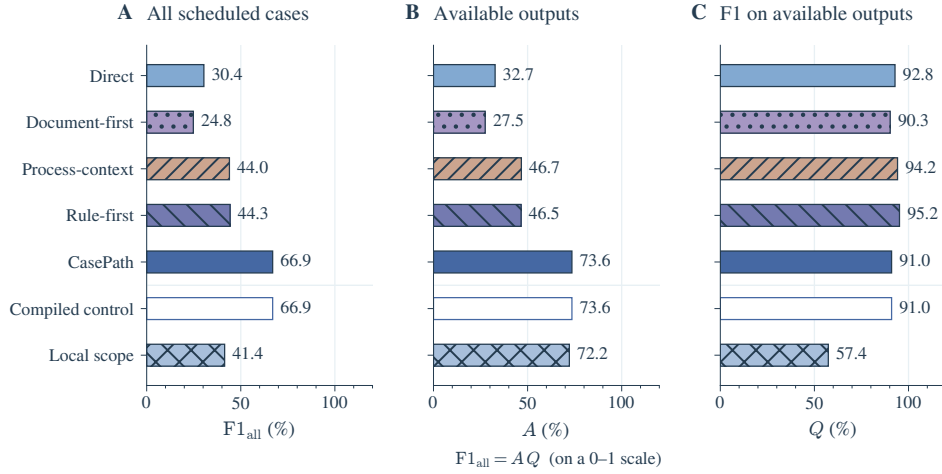


Figure 13: **CASEPATH produces a usable list more often; when lists exist, all pipelines do similarly well.** Over all 150 claims: (A) checklist F_1 with failures scored as zero; (B) availability A , the weighted share of claims with a scoreable list; (C) quality Q , the weighted mean F_1 of those lists. Overall F_1 is AQ .

C.3 WHAT FULL PROCESS SCOPE HOLDS, DROPS AND KEEPS

Full scope and Local scope are compared at CASEPATH’s saved judgments and differ only in the conditions inherited from the branches above each obligation’s own branch. Every original condition is evaluated against those judgments, never against the reference, with the three-valued logic of Appendix A.1. Entities known to be active enter a snapshot of the current case; unresolved ones stay on record but inactive. Branch conditions that describe other states of the case are not evaluated, and neither is the evaluator’s acceptance check on the whole output. The evaluator’s reference graphs, scoring functions and matching rules are used unchanged. Each control keeps its own graph and requests, bound to CASEPATH’s saved judgments for the same claim. All 110 snapshots and scores of Compiled control match CASEPATH.

On the 108 claims planned both ways, the two scopes make the same valid requests on every claim, 635 in all, and miss the same 93 of the 728 required documents. The whole difference lies in unnecessary requests, which fall from 810 to 17. Full scope could not have reduced the misses, since it can only hold or drop a request; coverage depends on the judgments and on what the process asks for. The only condition an obligation inherits is the dispute type (Section 4.2), and every removed request belongs to one of the claim’s other two types. Where the model left such a type unresolved, full scope holds its documents until that question is answered: 710 requests on 95 claims. Where the model judged it false, full scope drops its documents, whether Local scope requested them now or held them: 83 and 103 documents on 11 claims. Held documents therefore rise from 1,387 to 1,994 on these claims, and over all 110 of its lists CASEPATH holds 2,006.

By split, requests on these claims fall from 592 to 275 on development and from 853 to 377 on held-out claims, with 259 and 376 valid under both scopes. The output graphs shrink without losing a reference match. Pooled over both splits, candidate nodes fall from 8,931 to 3,261 and relations from 5,630 to 2,237 (Figure 8A), while the matches for nodes (2,331), relations (2,031) and requests (635) stay the same on every claim. These totals cover the 108 claims and count every item equally; Table 9 counts all 150 claims and weights dispute types equally.

Cutting each Local scope list at random to full scope’s length would keep fewer than half of the valid requests (Figure 8B). On claim i , let Local scope make ℓ_i requests, ℓ_i^{valid} of them valid, and let full scope make ℓ_i^{full} . Drawing ℓ_i^{full} of the Local scope requests uniformly without replacement keeps $\ell_i^{\text{valid}} \ell_i^{\text{full}} / \ell_i$ valid ones in expectation, so the figure reports, within each split,

$$\frac{\sum_i \ell_i^{\text{valid}} \ell_i^{\text{full}} / \ell_i}{\sum_i \ell_i^{\text{valid}}},$$

where the two claims with empty lists contribute zero to both sums. The calculation uses only the saved list sizes and makes no model calls; a learned selection method could do better than random deletion.

Table 9 gives the rates over all 150 claims, and two of them reduce to simpler ones. The documents with valid chains are exactly the necessary ones, so justified-request precision equals one minus the unnecessary fraction; and every active obligation carries the same criticality weight, so critical-evidence recall equals obligation completeness, the share of active obligations whose evidence is complete. The evaluator’s provenance-chain inheritance rate, which scores inherited source locations, is zero under both scopes, so the analysis does not show that the sources support every relation. If every claim on which either scope lacks an output counts as a loss, justified-request precision still gains +0.112 on development, +0.103 on held-out claims and +0.107 overall, while critical-evidence recall changes by −0.261, −0.289 and −0.278. These recall changes come entirely from the claims counted as losses: wherever both scopes have an output, recall is identical.

Table 9: **At the same judgments, full process scope raises precision for nodes, relations and requests and leaves recall almost unchanged.** Full is CASEPATH; Local is the Local scope control, which drops the conditions inherited from the branches above an obligation’s own branch. All 150 claims count, with failed outputs penalised and dispute types weighted equally. Higher is better for every rate; bold marks the higher precision where the two scopes differ. Compiled control matches Full exactly and is omitted.

	Development		Held out		All claims	
	Full	Local	Full	Local	Full	Local
Claims with output	44	44	66	64	110	108
<i>Output graph</i>						
Node precision	0.523	0.263	0.536	0.256	0.531	0.258
Node recall	0.542	0.542	0.544	0.526	0.543	0.532
Relation precision	0.676	0.378	0.679	0.368	0.678	0.372
Relation recall	0.374	0.374	0.381	0.370	0.379	0.372
<i>Requests</i>						
Justified-request precision	0.704	0.331	0.732	0.318	0.721	0.323
Critical-evidence recall	0.635	0.635	0.645	0.627	0.641	0.630
Valid / all requests	259/275	259/592	384/385	376/853	643/660	635/1,445

C.4 ALL SEVEN METHODS

Figure 13 factors each method’s checklist F_1 into availability A and quality Q . CASEPATH leads on availability because code assembles its output; on the lists that exist, its quality lies within the range of the four baselines. Local scope separates the two effects: code assembles its output too, so its availability is close to CASEPATH’s, but its lists keep the unnecessary requests that full scope removes, and its quality is 57.4%.

Table 10 gives each method’s scores by split, and Table 11 the document counts over all claims, including the documents each method holds until a condition is resolved. One recorded development

trace, selected before any target was scored, shows such a hold. Arrears are judged true, so the missing cure notice is requested; whether the property is the family home is unknown, so a notice to the spouse stays on hold. The plan has seven documents to request now and four on hold, and its next action asks the question that decides whether another obligation applies.

Counting every missing output against CASEPATH, including a baseline’s own failures, turns all conservative contrasts negative (Table 12): against each baseline, more than half of the held-out claims lack at least one of the two lists and so score -1 whatever the other list contains.

Table 10: **CASEPATH leads the pipelines in every column on both splits.** Document-request scoring over all claims of each split; a failed output scores 0 for F_1 and 1 for the unnecessary fraction. Dispute types weigh equally, then families within a type and claims within a family (Appendix C.2). The controls reuse CASEPATH’s judgments. Bold marks the best pipeline in each column.

Method	Claims with output		Checklist $F_1 \uparrow$		Unnecessary fraction $\downarrow$	
	Development	Held out	Development	Held out	Development	Held out
<i>Pipelines</i>						
CASEPATH	44	66	0.657	0.677	0.296	0.268
Direct	19	30	0.295	0.310	0.687	0.688
Document-first	17	24	0.247	0.249	0.732	0.737
Process-context	27	43	0.407	0.458	0.570	0.538
Rule-first	28	42	0.433	0.450	0.545	0.540
<i>Controls</i>						
Compiled control	44	66	0.657	0.677	0.296	0.268
Local scope	44	64	0.420	0.411	0.669	0.682

Table 11: **CASEPATH and its controls hold the most documents, and Local scope requests more than twice as many as CASEPATH.** Counts are summed over the lists of Table 10. Correct counts requested documents that are in the reference; for CASEPATH and Local scope these equal the valid requests of Table 9. Held counts documents that wait for a condition to be resolved. Precision and recall are the document-request scores over every claim, with failure penalties and weights, and are not ratios of the counts; for CASEPATH and Local scope they equal the justified-request precision and critical-evidence recall of Table 9. Bold marks the best pipeline in each rate column.

Method	Documents, summed over lists				Over all claims	
	Requested	Correct	Missed	Held	Precision $\uparrow$	Recall $\uparrow$
<i>Pipelines</i>						
CASEPATH	660	643	95	2,006	0.721	0.641
Direct	316	301	30	65	0.312	0.298
Document-first	247	237	39	139	0.265	0.236
Process-context	449	433	36	31	0.451	0.434
Rule-first	439	432	36	36	0.458	0.432
<i>Controls</i>						
Compiled control	660	643	95	2,006	0.721	0.641
Local scope	1,445	635	93	1,387	0.323	0.630

C.5 PROVENANCE AND REPLAY

Product work had exposed the case inputs of the held-out claims, and the prediction run ended without its complete audit of inputs and outputs, so we cannot prove that their reference labels were never accessed. The scoring step authenticates both complete sets of predictions before it reads any reference. The original scoring, the diagnosis of the condition symbols and document-request scoring each keep a separate record, and every record preserves the original failures. All reference containers have since been released unchanged under the recorded data licence.

Every Study B report can be replayed offline. The anonymous supplement holds all 1,050 prediction and failure records, the exact 150 reference containers, the original evaluator, the reporting code, the

Table 12: **Conventional contrasts favour CASEPATH; conservative contrasts, which count every missing output against it, do not.** Paired contrasts over the held-out claims (Appendix C.2); positive values favour CASEPATH, with the unnecessary fraction’s sign reversed. A conservative contribution is -1 whenever either list is missing.

CASEPATH against	Conventional		Conservative	
	Checklist F_1	Unnecessary fraction	Checklist F_1	Unnecessary fraction
Direct	+0.367	+0.419	−0.758	−0.744
Document-first	+0.429	+0.469	−0.788	−0.785
Process-context	+0.219	+0.270	−0.673	−0.657
Rule-first	+0.227	+0.272	−0.671	−0.650

source commitments and the document-request scorer. The replay rescores the saved outputs with no numerical tolerance, without regenerating them or calling the model. It reproduces the predeclared report of each phase, the full predeclared report, every row of document-request scoring and the descriptive report, and their canonical JSON matches exactly.

D THE RELEASED DATASETS AND SOFTWARE

Section 4 introduces the two datasets through Figures 3 and 4. For each dataset, this appendix describes a case and its reference, where the text comes from and what the data cannot test; the last subsection separates the released software from the method the paper evaluates. Each dataset’s README in `data/` is its datasheet, and a script checks every released file against recorded hashes. Code is under Apache 2.0, and the data are under CC BY 4.0 except quoted third-party text, which stays with its issuers (Tables 13 and 15).

D.1 STUDY A: 36 PAIRS OF THEFT CASES

A Study A case is a short English story of a household theft followed by nine statements, one for each branch condition. In the first case of a pair, all nine conditions are off; in the second, one of them is on, and nothing else changes. For example, “No stolen item is separately scheduled in the policy with an insured value of CHF 1,000 or more” becomes “A stolen watch is separately scheduled in the policy with an insured value of CHF 4,000.” Every method receives the same source passages and document catalogue (Tables 13 and 14).

The reference of a pair lists the documents its change adds, grouped into requirements; a requirement is the set of acceptable alternatives for one piece of evidence. Where the sources accept alternatives, the reference lists every acceptable combination, and Appendix B.1 scores a pair against the closest one. Code derived the references from a contract that an agent wrote from the sources. The contract records the decisions, conditions, facts, evidence and documents of the theft process that the sources support, with a verbatim quotation for every decision and condition. Two amendments, both made before the held-out pairs were read, turned “receipt and valuation” into “receipt or valuation” and added the bicycle purchase voucher. Besides the cases and references, the release holds the generator of the cases, the derivation of the references, the recorded checklists of the four methods and CASEPATH’s frozen representation, which is model output and not part of the reference.

The cases are invented, and no person, policy or claim in them is real, although the thresholds and periods in the condition statements come from the sources. Each passage is a short verbatim excerpt, attributed to its document in the release (Table 14). Swiss federal enactments such as the VVG are not protected by copyright. In the frozen files, eight passages carry the label of a document that does not contain them, although each is verbatim in another document of the corpus. The release corrects these labels in its source notes and leaves the frozen files unchanged. The release also documents two stale convenience fields that the scorer does not read.

What the dataset does not cover. No legal expert reviewed the contract behind the references. Because a case switches on at most one condition, interactions between conditions are not tested. Every reference change is an addition, so a correct withdrawal cannot be measured. Only one

Table 13: **The 36 pairs cover nine branch conditions, and each records the change its one fact requires.** Counts over all pairs.

	Count	Detail
<i>Design</i>		
Pairs	36	9 development, 27 held out
Cases	72	40 distinct texts; within a story, the nine cases with every condition off are identical
Branch conditions	9	three groups: stolen property, events after the claim, settlement and recourse
Stories	4	claim settings, the first for development
Words per case	141–155	English
<i>Reference</i>		
Required changes	44	11 development, 33 held out; all additions
Pairs with alternatives	4	one requirement: a receipt or an expert valuation
Condition states	648	nine per case, fixed by construction
<i>Shared inputs</i>		
Source passages	197	from 7 documents (Table 14)
Document catalogue	33	document types, all named in the sources
<i>Release</i>		
Methods recorded	4	each with both checklists of all 36 pairs
Scorer		frozen; recomputes every Study A result offline
Licence		data CC BY 4.0 except the quoted policy terms, claim form and model conditions; code Apache 2.0

Table 14: **The 197 passages are verbatim excerpts from seven public documents.** Three passages appear word for word in more than one document, so the counts sum to 201. The release gives the URL and file hash of each document.

Document	Edition	Passages
<i>Federal law</i>		
Insurance Contract Act (VVG), Arts. 38–41a, 45 and 46 (Swiss Confederation, 2024)	1 January 2024	26
<i>Household policy terms</i>		
CSS Versicherung AG (2024), household contents	02.2024	67
AXA Versicherungen AG (2021)	10.2021	17
Generali Allgemeine Versicherungen AG (2026)	06.2026	25
Simpego Versicherungen AG (2024)	12.2024	9
<i>Claim form</i>		
CSS Versicherung AG (2026), household contents and building	08.2026	19
<i>Industry model conditions</i>		
Schweizerischer Versicherungsverband SVV (2022), household contents	01.04.2022	38
Sum over documents		201
Distinct passages		197

requirement has alternative routes. The held-out pairs use new claim settings for the same nine conditions and were read once, under the freeze of Appendix B.1; they have been public since and remain a fresh test only for work that keeps them out of development.

D.2 STUDY B: 150 TENANCY CLAIMS

An agent sees two files per claim. The intake packet holds the submission’s channel, date and language, the customer message as sent and any attachments, embedded so that each claim file stands alone. The source registry lists the 136 to 140 units an agent may cite: the 101 entries of the three shared process templates, 19 passages of the Swiss Code of Obligations on tenancy (Swiss

Confederation, 2026), 15 to 19 proposition-sized spans of the message and an inventory of the attachments.

The reference is the generator’s own state for the case, and no method’s output was used to write it. It records the dispute type, the scenario flags that decide each branch and a process graph whose typed relations link decisions, facts, evidence capabilities, documents, steps and outcomes. It also records the order of the steps, the evidence obligations with the expected state of each document, the process state at intake, the next action and three or four source locators for each grounded concept (Table 15). *Conditional* is the reference’s name for the state of a document that should be held until a condition is resolved.

Table 15: The 150 claims cover three dispute types in two languages, and each carries a full structured reference. Counts over all claims; ranges are per claim.

	Count	Detail
<i>Intake</i>		
Claims	150	60 development, 90 held out
Dispute types	3	defects, lease terminations and rent increases, 50 claims each
Families	28	11 development, 17 held out; no family crosses the split
Languages	2	English 75, Swiss Standard German (de-CH) 75
Channels	4	email 70, portal 46, chat 24, phone note 10
Attachments	57	in 45 claims: 47 PDF notices, demands and forms, 10 photos
Words per message	138–209	median 179
<i>Reference</i>		
Concepts (nodes)	7,800	46–55; includes 600 that an output must not contain
Relations	8,100	47–58; includes 600 that an output must not contain
Branch conditions	750	3–7
Obligation chains	1,397	8–10, each from decision to fact, evidence and document
Order constraints	3,450	20–25
Document states	1,397	missing 961, conditional 395, provided but insufficient 41
Next action	150	process step 100, document request 35, escalation to an external specialist 15
<i>Release</i>		
Evaluator		33 metrics and 8 acceptance gates per claim
State-change track	48	variants: 12 base claims, each updated in four ways
Licence		data CC BY 4.0 except the cantonal form templates; code Apache 2.0

The released evaluator scores 33 metrics over process structure, evidence, grounding and calibration, and accepts a claim’s output only if it passes eight gates, among them valid paths, complete obligations, correct document states and no forbidden elements. The reference output of every claim passes, and empty and request-everything outputs fail. The gates evaluate an output’s branch conditions against the reference scenario, so an output must state them in the scenario’s flags. The outputs of Section 4.2 used the compiler’s own symbols instead, and the evaluator accepted none of them. Appendix C.1 explains this failure and Appendix C.2 the document-request scoring used in its place.

The state-change track updates each of 12 base claims in four ways: it adds a sufficient document, makes a branch fact false, leaves that fact unresolved or resolves every active fact. It scores nine endpoints, among them whether each request moves between requested now, held and dropped, and whether the unaffected part of the graph stays unchanged. The 24 development variants ship with their references. The references of the 24 held-out variants were regenerated with the released track code from the held-out claim references, and each matches the hash committed in the package.

The claims, messages, parties, images, process templates and references are synthetic. Two kinds of real material appear: the 19 legal passages, which Swiss copyright law does not protect, and the official Canton of Zurich forms on which 34 of the 47 PDF attachments are filled in. Every evaluated message named the intake mailbox of a real insurer as its recipient, in the raw message header that the pipelines did not read. The release replaces the address with a fictional one of the same byte length, and a record binds each of the 300 changed files to the hash of its evaluated original, so the original bytes can be checked once the name is disclosed after review. The 60 development references ship

with the benchmark package, and the 90 held-out references are released beside it, each byte-identical to the file whose hash `cohort.json` committed before the evaluation.

What the dataset does not cover. All claims are set in one canton. The reference captures each case at intake, and change over time appears only in the one-update variants of the state-change track. Each obligation has one acceptable document, so route choice is not tested. Images carry no labelled regions, and no field records amounts, liability or coverage. The next action, including escalation to a specialist, is labelled but not scored by the released evaluator, and it varies only among defect claims: every termination and rent-increase claim expects the step that preserves the challenge or extension deadline, and all 35 document requests and 15 escalations fall in defect claims. The held-out inputs were seen during product work and their references were released after the evaluation of Section 4.2, so the held-out split is not a fresh test (Appendix C.5). It remains a family-disjoint test split for anyone who keeps it out of training and tuning.

D.3 THE RELEASED SOFTWARE

The workbench of Section 3 is a local, single-user application released with this paper. A browser interface runs over a service that stores every change to a claim as an event in a hash-chained journal. A claims handler opens any of the 150 claims of Study B and reads the original message and attachments beside the active process step, the evidence to check and the next action. The handler can then record, correct, export and replay the handling state; nothing is sent to a customer. The workbench also runs its own review, which places a claim in one of the three process templates and runs six agent roles in a fixed order; only the process role may change claim state. In the released version this review is deterministic and makes no model calls. It is product behaviour, and the results of Section 5 do not carry over to it. Table 16 separates the method, the benchmark tools and the product.

Table 16: **The paper measures the method; the software hosts it.** What each part of the release is and what supports it.

Part	What it is	Evidence
<i>Method</i>		
Study A	Condition interpreter and planner of Equation 3, installed in the service as a separate endpoint	Offline replay of all 72 cases matches
Study B	Planner of Equations 1 and 2, shown on an authored teaching example	Section 5.2; 45 teaching states re-executed; no parity claim
<i>Benchmark tools</i>		
Study A	Scorer and explorer	Appendix B
Study B	Evaluator, document-request scorer and state-change track	Appendix C
<i>Product</i>		
Workbench	Interface, journal, agent review, correction and replay over the 150 claims	Demonstrates handling mechanics only

For Study A, a verifier checks that the installed method makes the recorded decisions. It loads the installed method pack and the service, replays all 72 cases with the recorded condition judgments and compares condition values, document sets and next actions. It also checks all 36 paired changes with the service’s comparison function and confirms that every displayed quotation occurs exactly in its source passage. Every check passes. For Study B the workbench is a separate implementation, and we make no parity claim. The supplement also holds a teaching example of the Study B planner, an invented repair policy whose 45 authored states `replay_teaching.py` re-executes and checks. It is labelled as teaching material and kept apart from the evidence.

A live run of the service needs provider access and covers theft claims only. A copy downloaded from the anonymous mirror does not start the workbench, because the mirror rewrites strings in files that the launcher checks; the dataset verifiers and the Study A replay are unaffected.

E EXTENDED RELATED WORK

Section 2 places CASEPATH among the closest work in a few lines. This appendix first explains why none of the cited systems could be a baseline and compares CASEPATH with the two closest precedents. It then takes each line of work in turn, saying what its systems take and return and where they differ from CASEPATH.

E.1 WHY NO CITED SYSTEM IS A BASELINE

None of the cited systems could run unchanged on our task (Table 17). Our cases need a list of documents to request, and we score each request. The workflow benchmarks and agents return tool calls, environment actions or a final database state, and no cited system has public code that takes a process and a case and returns such a list. Some also need inputs our cases lack. StateFlow’s authors note that it “requires humans to have a good understanding of a given task and build the model and prompts” (Wu et al., 2024), and Compile-then-Page takes machine-readable constraints and was not tested on procedures written in natural language. Compile-then-Page (“Artifacts will be released”) and the evidence ledger had no public code when we checked, and the Procedural Graphs paper links no official code.

Instead, our baselines test what these systems share on the model side: the model reads the rules or the process and chooses the requests itself, with the same model, sources and cases as CASEPATH. The last column of Table 17 names the closest baseline for each system. We did not rebuild StateFlow’s state transitions, the self-evolving graph of Procedural Graphs, the paging runtime of Compile-then-Page, the evidence ledger or a guardrail filter over a baseline’s list, so our results do not show that CASEPATH beats these systems.

E.2 THE CLOSEST PRECEDENTS

The closest precedent is the evidence ledger of Kim (2026). In it, branch conditions activate knowledge obligations, a model judges semantic warrant and code manages closure. Its obligations are what to retrieve to answer one query, and a model plans them for each query; ours come from the process, which is the same for every case. Its condition nodes keep a branch’s obligations inactive until the condition is resolved, and a condition-evaluation tool call must resolve each one before the answer is final. Our conditions are judged from the case and may stay open in the plan: the request is held, and the condition becomes a question for the claimant. The paper describes no conditions that an obligation inherits from enclosing branches, and in Study B such conditions remove more than half of the requests. The ledger retrieves from a knowledge base, asks no person for documents and keeps no alternative routes. Its Warrant Judge checks whether a retrieved passage answers the query, a judgment close to ours on whether a document on file is adequate. The paper analyses deployment traces from one university knowledge base and releases no benchmark or code.

Yu et al. (2026) compile the machine-readable constraints of standard operating procedures into pseudo-code whose rule routines return whether a constraint holds and the evidence for it, and a runtime shows the model only the active frame. The model still gives every verdict and chooses every tool call, and the authors call this enforcement “deliberately soft”. In CASEPATH, once the process is built, the model only judges the conditions and, in Study B, the documents on file. Code combines these judgments, so every request traces to an obligation that the process makes active.

In planning document requests, an obligation can apply before its evidence may be requested, a document on file can be inadequate and several documents may have to be assessed together. The representation also keeps alternative routes, and Study A shows what requesting all of them costs (Table 7). Neither study tests timing, route choice or joint assessment (Appendix A).

E.3 RULES, PROCESSES AND PROCEDURE EXTRACTION

CASEPATH builds on keeping rules apart from the state of the case they apply to. LegalRuleML records norms and their sources (OASIS LegalRuleML Technical Committee, 2021), and Catala makes legislative computation executable (Merigoux et al., 2021). DMN formalises decisions, and CMMN represents case work that conditions activate (Object Management Group, 2024; 2016). Guard-Stage-Milestone models describe decision- and knowledge-intensive processes around business

Table 17: **No cited process-aware system runs unchanged on our cases.** In the last column, (A) and (B) mark Study A and Study B.

			On our cases	
System	Returns	Would need	Why not run	Closest test
<i>Workflow benchmarks</i>				
FlowBench (Xiao et al., 2024)	Tool calls in a dialogue	A workflow as text, code or flowchart, and tools	A benchmark	Graph context (A); every B baseline has the process in its prompt
τ -bench (Yao et al., 2025)	Tool calls and replies	The policy as system prompt, tools and a simulated user	A benchmark	Direct (A), with the policy text in its prompt
<i>Process-aware agents</i>				
StateFlow (Wu et al., 2024)	SQL, shell or household actions	A state machine and prompts built by hand for each task	A state machine for our processes would be our own design	Process-context (B), which resolves the process before choosing
Procedural Graphs (Lu et al., 2026)	The next action, with advice from the active node	A tool call to find the active node (a one-step task stays at the Start node); scored runs and a validation split to evolve the graph	One list per case, no tool calls; no official code	Graph context (A): the graph in the prompt without the guidance model
Compile-then-Page (Yu et al., 2026)	Tool calls on SOPBench, scored on the call sequence and final database state	Machine-readable dependency constraints for each user goal	Code not released	Every B baseline has the compiled process in its prompt; the paging runtime is not tested
<i>Evidence planning</i>				
Evidence ledger (Kim, 2026)	An answer to a knowledge-base query	Obligations a model plans for each query, and a retrieval tool	No code or benchmark	Evidence-first (A), Document-first (B): evidence needs first, no code-kept states
<i>Guardrails</i>				
ShieldAgent, GuardAgent (Chen et al., 2025; Xiang et al., 2025)	Allow or deny for another agent’s action	Another agent’s actions to check	Could only filter a baseline’s list	None
<i>Process builders</i>				
PAGED, BREX, Guideline2Graph, Graus (Du et al., 2024; Yang et al., 2026; Kilic et al., 2026; Graus, 2026)	A procedural graph, rule flow or decision model	Source documents. Generation takes no case; Graus runs the generated models on preset test scenarios	They build a process, the step CASEPATH’s agents do in Study A	Rule-first (B), which derives and runs its own rules
<i>Rule engines and products</i>				
DMN or Catala rule engine (Object Management Group, 2024; Merigoux et al., 2021)	Decisions on typed inputs	Typed facts, while our cases are free text	On the model’s judgments it would compute what Compiled control computes	Compiled control (B): CASEPATH’s decisions as one flattened formula, matched exactly
ServiceNow document lists (ServiceNow, 2026)	A document list per claim service definition, set when a case is opened	The commercial platform	Closed software; reads only the service definition	None

artifacts (Vaculín et al., 2011), and DCR models of law check whether a business process complies with the law (López et al., 2020). Proactive enforcement deduces the actions that a policy of provisions and obligations requires and causes them in the target system (Basin et al., 2024). None of these reads the facts from a free-text case. LegalRuleML only encodes rules, and the DCR work checks process models rather than cases; the systems that run rules take the facts as structured data or, in proactive enforcement, as the events of the monitored system. Commercial insurance software picks a claim’s document list from its service definition when the case is opened (ServiceNow, 2026). The documented step reads only that definition, holds no request and asks no question.

Other work builds the process itself. PAGED is a benchmark for extracting procedural graphs from documents (Du et al., 2024), and WorFBench evaluates whether a model can generate a workflow for a task (Qiao et al., 2025). BREX is a benchmark for extracting business-rule flows from business documents, and its ExIde framework includes executable grounding through generated pseudo-code (Yang et al., 2026). Guideline2Graph converts clinical guidelines into executable decision graphs that keep their provenance (Kilic et al., 2026), and Graus (2026) tests whether structured intermediate representations help a model generate executable decision models from legal text. CANONIC decides whether content may enter a corpus the way a compiler decides whether a program is well formed (Hadley, 2026); it checks content against a grammar and has no case to plan for. CASEPATH also needs a process, which agents built from the sources in Study A and code compiled from templates in Study B, and then uses it to decide what one case still needs.

Neuro-symbolic systems split the work between a language model and an exact engine. The model translates a problem into a formal representation, and the engine decides: Logic-LM hands a symbolic formulation to a solver (Pan et al., 2023), and Guan et al. (2023) have a model build a PDDL domain model that a sound planner then uses. CASEPATH applies this split to choosing actions in a workflow. Its process is built once for all cases and is the exact engine, and the model judges each case’s conditions. Our Rule-first baseline is the variant in which the model writes the rules and follows them itself; in Study A, agents write the rules and code runs them.

E.4 PROCESS-AWARE AGENTS AND GUARDRAILS

Workflow benchmarks and process-aware agents give the model a workflow or policy to act on. FlowBench evaluates workflow-guided planning: it gives the agent a workflow as text, code or a flowchart and scores its tool calls and whether each session meets the user’s goals (Xiao et al., 2024). In τ -bench the domain policy is the agent’s system prompt, and a task succeeds when the final database state matches the goal and the agent’s replies contain the required information. In the original study the best agent, gpt-4o, succeeded on 61% of retail tasks and 35% of airline tasks (Yao et al., 2025). StateFlow separates process grounding, a state machine over the task, from the actions taken in each state (Wu et al., 2024), and ReAct interleaves reasoning and action without a process model (Yao et al., 2023). Procedural Graphs start an agent at a Start node and move it to the node that matches its last tool call, falling back to the whole graph only when no node matches. A guidance model turns the graph around the active node into advice that biases the next action without dictating it, and a model refiner edits the graph by contrasting failed and successful trajectories (Lu et al., 2026).

In FlowBench, τ -bench, Procedural Graphs and StateFlow the model chooses the actions, apart from a few fixed steps in StateFlow, and the workflow or policy informs that choice. Procedural Graphs and StateFlow also track one current node or state, while CASEPATH evaluates every obligation at once and, under full process scope (Study B), every branch that encloses it.

Guardrail agents bring rules to language-model agents. ShieldAgent compiles policy documents into probabilistic rule circuits and verifies an agent’s action trajectory against them (Chen et al., 2025), and GuardAgent turns safety requests into guardrail code that checks each proposed action (Xiang et al., 2025). Both judge actions that another agent chose. ShieldAgent also explains each violated rule and suggests countermeasures. Since neither plans the actions, they would not add a request that the other agent left out or ask about an open condition.

E.5 INFORMATION SEEKING AND GROUNDING

Information-seeking systems decide what to look up or ask. InfoGatherer uses evidential uncertainty to choose what to retrieve and what to ask (Taranukhin et al., 2026), and Self-RAG learns when to retrieve (Asai et al., 2024). MediQ has a model hold back a diagnosis and ask follow-up questions when it is not confident (Li et al., 2024), and Ask-before-Plan predicts when a planning agent needs clarification (Zhang et al., 2024). Grounding work supports answers with evidence. GopherCite answers with quoted evidence (Menick et al., 2022), and chain-of-verification checks a draft answer with verification questions (Dhuliawala et al., 2024). GraphCompliance aligns policy and context graphs for regulatory assessment (Chung et al., 2025), and CLER is a benchmark for reasoning about evidence in Chinese litigation (Bi et al., 2026). API-Bank and BFCL evaluate function calling in tool use (Li et al., 2023; Patil et al., 2025). None of these systems has a process that says which requests a case calls for, and those that seek information retrieve or ask in order to answer a question or make a plan. CASEPATH asks when a condition that decides whether an obligation applies is still unresolved and requests documents when an active obligation’s fact is still open. It quotes the case for each condition it judges and keeps a chain from source to document for every request.

E.6 VALIDITY OF THE EVALUATION DESIGN

Shortcut and validity research shaped our controls and outcomes. Work on partial-input baselines, annotation artefacts and shortcut learning motivates our input-free and wrong-condition controls (Kaushik & Lipton, 2018; Poliak et al., 2018; Gururangan et al., 2018; McCoy et al., 2019; Sugawara et al., 2018; Geirhos et al., 2020; Bowman & Dahl, 2021; Yuan et al., 2024). Code derived Study A’s references from a hash-bound contract that an agent built from the sources, and code scored the outputs, which avoids judge biases such as preference leakage, in which a judge model favours models trained on data from a related model (Li et al., 2026). Research on evaluation validity (Thomas et al., 2026), the finding that few papers on legal judgment prediction actually predict court decisions (Medvedeva & McBride, 2023) and the critique that legal benchmarks start from inputs that legal experts have already prepared (Lou & Shin, 2026) motivate our choice of document changes and requested documents as outcomes.